

TECHNICAL DOCUMENTATION · CONFIDENTIAL

AgentEdge

Every file, how it works, how the AI model is designed

Repository: `austin2134-cell/sportsagent`

By `EdgeSportsMedia`

Generated 2026-07-06

Table of Contents

AgentEdge

- Complete Technical Documentation
- Chapter 1: What This Is
 - The two pipelines
- Chapter 2: System Architecture
 - Stack
 - Request/data flow for one scheduled agent scan
- Chapter 3: How the AI Model Is Designed
 - Persona and mandate
 - Sizing model: fractional-Kelly by edge gap
 - EV-first pricing, no juice ceilings (except soccer)
 - World Cup / soccer: a fully separate rule set
 - Data integrity and source hierarchy (Section 2)
 - Sharp vs. public interpretation
 - Output contract
 - Confidence tiers and voice
 - Prompt caching — why the model is cheap to run continuously
 - Model and token configuration
- Chapter 4: Per-User Agent Architecture — backend/agent/
 - backend/agent/__init__.py
 - backend/agent/bankroll.py
 - backend/agent/bankroll_backfill.py
 - backend/agent/kernel.py
 - backend/agent/memory_store.py
 - backend/agent/post_grade.py
 - backend/agent/prompt.py
 - backend/agent/provision.py
 - backend/agent/sports.py
 - backend/agent/unit_tracker.py
- Chapter 5: Data & Intelligence Layer — backend/esm/
 - backend/esm/__init__.py
 - backend/esm/system_prompt.py
 - backend/esm/action_network_client.py
 - backend/esm/api_budget.py
 - backend/esm/claude_config.py
 - backend/esm/config.py
 - backend/esm/market_intelligence.py
 - backend/esm/odds_client.py
 - backend/esm/snapshot_cache.py
 - backend/esm/soccer_odds.py
 - backend/esm/split_guidance.py
 - backend/esm/stats_client.py
- Chapter 6: Application Services — backend/services/

- backend/services/__init__.py
- backend/services/agent_runner.py
- backend/services/card_store.py
- backend/services/daily_esm_card.py
- backend/services/grader.py
- backend/services/mailer.py
- backend/services/sheets_sync.py
- backend/services/social.py
- backend/services/splits_sync.py
- backend/services/units.py
- backend/services/world_cup_card.py
- Chapter 7: Core Backend, Learning Loop, Workers & Scripts
 - backend/main.py
 - backend/auth.py
 - backend/database.py
 - backend/run_daily_esm_card.py
 - backend/run_world_cup_card.py
 - backend/sync_sheets.py
 - backend/learning/__init__.py
 - backend/learning/memory.py
 - backend/workers/__init__.py
 - backend/workers/market_poller.py
 - backend/scripts/backfill_bankroll.py
 - backend/scripts/correct_wc_card_dnb.py
 - backend/scripts/production_setup.py
 - backend/Dockerfile
 - backend/railway.toml
 - backend/requirements.txt
 - backend/.env.example / backend/.env.template
 - backend/.python-version / backend/runtime.txt
- Chapter 8: Frontend Application — frontend/
 - app/agent/page.tsx
 - app/auth/callback/route.ts
 - app/dashboard/page.tsx
 - app/globals.css
 - app/history/page.tsx
 - app/layout.tsx
 - app/login/page.tsx
 - app/page.tsx
 - app/preferences/page.tsx
 - app/register/page.tsx
 - app/reset-password/page.tsx
 - app/setup/page.tsx
 - components/AuthHashHandler.tsx
 - lib/api.ts
 - lib/auth-routing.ts
 - lib/env.ts
 - lib/supabase.ts / lib/supabase/server.ts

- next.config.js / package.json / postcss.config.js / tailwind.config.ts / tsconfig.json
 - public/card-preview.html
 - .env.example
- Chapter 9: Database Schema & Migrations — supabase/
 - supabase/schema.sql
 - supabase/migrations/001_agentedge.sql
 - supabase/migrations/002_agentedge_idempotent.sql
 - supabase/migrations/003_platform_memory.sql
 - supabase/migrations/004_market_splits.sql
- Chapter 10: CI/CD, Automation & Operational Docs
 - .github/workflows/production-setup.yml
 - .github/workflows/sync-sheets.yml
 - .github/workflows/wc-card.yml
 - docs/CUSTOM_DOMAIN.md
 - docs/sessions/README.md
 - scripts/run_migration.py
 - scripts/set_railway_env.sh
 - railway.toml (repo root)
 - .gitignore
- Chapter 11: Test Coverage
 - backend/tests/test_claude_config.py
 - backend/tests/test_dual_brain_memory.py
 - backend/tests/test_grader_soccer.py
 - backend/tests/test_learning_hooks.py
 - backend/tests/test_market_intelligence.py
 - backend/tests/test_sheets_sync.py
 - backend/tests/test_soccer_dnb_odds.py
 - backend/tests/test_tier1_learning.py
 - backend/tests/test_unit_tracker.py
 - backend/tests/test_units.py
- Chapter 12: Environment Variables & Deployment Reference
 - Backend environment variables
 - Frontend environment variables
 - Production URLs
 - Running locally

Chapter 1: What This Is

AgentEdge (by EdgeSportsMedia) is a per-user sports betting AI agent platform. Each user provisions their own agent with a starting bankroll, sport preferences, bet-type preferences, and a risk profile. Once provisioned, the agent runs continuously in the background — polling betting markets, writing observations to a live "reasoning feed," tracking speculative hypotheses before acting on them, recommending sized positions (bets), and learning from the outcomes of bets it has already made.

The product evolved from an earlier, simpler version called **EdgeBet** (v1), which ran a single AI call once per day per user and produced a static "daily card" of up to five recommended wagers. That system is still present in the codebase as the "ESM" (Edge Sports Media) daily-card pipeline — it now runs *alongside* the newer, continuously-scanning agent architecture (v2) rather than being replaced by it. The legacy name "EdgeBet" may still appear in some deployment URLs and default constants.

At its core, the whole platform is a thin, carefully-engineered orchestration layer around a single large, static **system prompt** (the "ESM Framework") that turns Claude into a persona: a professional sports gambler whose only goal is week-over-week bankroll growth. Everything else in the codebase — the odds clients, the injury-data clients, the market-intelligence/line-movement analytics, the bankroll math, the memory/learning tables — exists to feed that one Claude call the highest-quality, most current context possible, and then to safely parse, validate, size, and persist whatever comes back.

The two pipelines

Pipeline	Trigger	Scope	Output
Continuous Agent (v2)	Runs every ~30–180 min for every provisioned, active user	Major-league sports only (MLB/NBA/NHL/NFL) — World Cup is intentionally excluded	Live reasoning feed (<code>agent_episodes</code>), tracked hypotheses, learned beliefs, and sized bet positions written directly to <code>bets</code> / <code>cards</code>
Daily Card (v1 "ESM")	Once per day, scheduled (MLB card ~9:35 AM MT, World Cup card ~8:50 AM MT)	Major-league "ESM" card + a dedicated World Cup 2026 card pipeline with its own market rules	A structured "daily card" (<code>cards</code> table) with up to 5 official plays, leans, quick reads, and pass notes; optionally emailed and formatted as a Twitter/X thread

Both pipelines share the same underlying primitives: the same odds/stats clients, the same bankroll/unit-sizing math, the same base system prompt, the same grading and learning-memory infrastructure, and the same Supabase database.

Chapter 2: System Architecture

Shared layer:	market poller (every ~6h)	→ market_snapshots (odds cache)
	Action Network splits sync	→ market_splits (public/sharp betting %)
Per-user:	agent scan (every 30–180 min)	→ agent_episodes / agent_hypotheses / agent_beliefs
/ bets		
Daily:	World Cup card (~8:50 AM MT)	→ cards.raw_card.world_cup
	MLB/ESM card (~9:35 AM MT)	→ cards.raw_card.esm
Grading:	ESPN box-score grader (~9:15 AM MT)	→ bets.result / bets.units_result
Learning:	memory refresh (post-grade)	→ agent_memory (per-user) / platform_memory (cross-user)
Onboarding:	/setup wizard	→ agent_instances + preferences + seeded
agent_beliefs		
UI:	/agent cockpit (live feed) /dashboard (today's card) /history (bet log)	
/preferences		

Stack

Layer	Technology
Backend	FastAPI (Python 3.12), deployed on Railway via <code>uvicorn</code> , Dockerized
Frontend	Next.js 14 (App Router), Tailwind CSS, <code>lucide-react</code> , deployed on Vercel
Database / Auth	Supabase (PostgreSQL + Auth + Row-Level Security)
AI	Anthropic Claude (<code>claude-sonnet-4-6</code> by default), via the official Python SDK, with ephemeral prompt caching
Scheduling	APScheduler (<code>AsyncIOScheduler</code>) inside the FastAPI process, backed up by GitHub Actions cron workflows
Odds data	Dual-provider: SportsGameOdds (SGO) as primary, The Odds API (TOA) as fallback / World-Cup-only source
Sports/injury context	ESPN's public (undocumented) API — no API key required
Betting-splits data	Action Network's public betting pages, scraped via their embedded Next.js data blob
Delivery	HTML email via SMTP or SendGrid; Twitter/X thread text formatting; Google Sheets export via <code>gsread</code>

Request/data flow for one scheduled agent scan

- `workers/market_poller.py` (or the scheduler directly) has already refreshed `market_snapshots` for all in-season sports, respecting API quota via `esm/api_budget.py`.
- `agent/kernel.py::run_agent_scan()` loads the user's `agent_instances` row, `preferences`, and `profiles` row.
- It resolves the user's sport preferences to odds-API sport keys (`agent/sports.py`), excluding World Cup (handled separately), and pulls a filtered market snapshot — cache-first (`esm/snapshot_cache.py`), live-API fallback (`esm/odds_client.py`).

4. It builds ESPN injury/scoreboard context (`esm/stats_client.py`) and market-intelligence signals (line movement, steam, public/sharp splits — `esm/market_intelligence.py`).
5. It gathers the agent's own memory: learned `agent_beliefs` , watched `agent_hypotheses` , recent `agent_episodes` , and 90-day performance history (`learning/memory.py`).
6. It builds the full system prompt (`agent/prompt.py` , which wraps the shared `esm/system_prompt.py`) and a large user-turn message containing all of the above, and calls the Anthropic API with the system block marked for ephemeral prompt caching.
7. Claude's JSON response is parsed and fanned out: new observations/hypotheses/belief updates are written to memory tables, and any recommended positions are validated against remaining daily unit exposure and persisted to `bets / cards` .
8. `agent/unit_tracker.py` re-syncs "units at risk" for the day, and `services/sheets_sync.py` mirrors the new bet(s) to Google Sheets if configured.

The **World Cup pipeline** (`services/world_cup_card.py`) follows a parallel but separate path with its own tournament-specific system-prompt rules (3-way vig removal, Draw-No-Bet juice-ceiling enforcement, altitude/host-nation/dead-rubber situational filters) and its own persistence path, merged into the same `cards` row under a different JSON key (`raw_card.world_cup` vs. `raw_card.esm`) so the two pipelines never clobber each other.

The next-morning **grading pipeline** (`services/grader.py`) pulls final scores from ESPN box scores, resolves W/L/P outcomes (including soccer-specific market resolution for DNB/3-way ML/BTTS/totals), and triggers `agent/post_grade.py` to update bankroll and close the loop on the originating episode in the agent's memory feed — which is what lets the *next* scan's prompt include "Lesson: Loss (-2.0u). Review thesis quality on late scratches." style context.

Chapter 3: How the AI Model Is Designed

This is the heart of the platform: a single ~550-line system prompt (`backend/esm/system_prompt.py` , the `ESM_SYSTEM_PROMPT` constant) that is sent, verbatim, as the `system` block on every Claude API call the platform makes — whether that's a continuous agent scan, the daily MLB/ESM card, or the World Cup card. Per-user personalization (bankroll, unit size, risk level, sports/bet-type preferences, memory) is layered *on top of* this shared base by `backend/agent/prompt.py` , never replacing it.

Persona and mandate

The prompt casts Claude as "**the ESM (Edge Sports Media) Betting Agent — a professional sports gambler operating autonomously.**" It states plainly: *"This is not a content exercise. This is survival."* The agent is explicitly told it runs continuously/daily to analyze live market data and output a structured card, and that its overriding, highest-priority mandate (Section 0, "PROFESSIONAL SURVIVAL MANDATE") is **week-over-week profitability and bankroll protection** — not daily win rate. *"A losing day is variance. A losing week is a warning. A blown bankroll ends the game."*

Key behavioral rules baked into that mandate:

- **Never chase losses.** If injected performance memory shows a losing streak, the model must tighten filters and cut size — never loosen up to "get back to even."
- **0-play days are valid.** Passing on a weak slate protects capital; grade the card D/F and walk away rather than force plays.
- **Market-level learning.** If memory shows a specific market (e.g. pitcher strikeout overs) losing consistently, avoid it until the data recovers — this overrides any optimism about today's specific matchup.
- **Volume discipline.** *"2 sharp plays a day over 200 days beats 5 mediocre plays a day over 200 days every time."* Hard cap of 5 official plays/day; never force 5 just to fill the card.

Sizing model: fractional-Kelly by edge gap

Stake sizing is **not** classic bankroll-percentage-of-Kelly-fraction math computed from odds alone — it's a discrete, four-tier system keyed off the **edge gap** (true probability minus implied probability), stated identically in both Section 0 and Section 4 of the prompt:

Edge gap	Unit size
Strong (10%+)	2.5 – 3 units
Solid (5–9%)	2 units
Moderate (2–4%)	1 – 1.5 units
Thin (<2%)	Lean only — never an official play

On weak slates (grade C or below) or during a losing streak, all unit sizes on the card are reduced by an additional 0.5u. Same-game parlays are capped at 1.5 units total regardless of individual leg sizing. The dollar value of "1 unit" itself is computed separately and dynamically by `agent/bankroll.py` as a percentage of the user's *current* (not starting) bankroll — see Chapter 4 — so the Kelly-tier table above scales automatically as the user's bankroll compounds up or down.

EV-first pricing, no juice ceilings (except soccer)

The prompt is explicit that **edge is the only valid filter** — **price level alone never disqualifies a play**: "*Juice level alone never disqualifies a play — mispriced heavy favorites are real edges.*" Every play must state both `implied_prob_pct` and `true_prob_pct` so the edge gap is auditable, and heavier juice requires a wider probability gap to justify the play, since estimation error compounds at extreme prices. The one explicit exception is the World Cup pipeline, which enforces a hard **-150 juice ceiling** on Draw No Bet plays (see below) — a market-specific integrity rule, not a general policy.

World Cup / soccer: a fully separate rule set

Section 1 of the prompt carves out an unusually detailed sub-framework for the FIFA World Cup 2026, because three-outcome (win/draw/loss) markets need fundamentally different math than the two-outcome US major-league markets the rest of the prompt assumes:

- **Historical baselines are context, never selectors** — group-stage games average ~2.0–2.4 goals (~57% Under 2.5 historically), DNB on clear favorites often hits 65–75%, and the model is told explicitly not to default to a market type just because of these averages; every match must be evaluated on its own merits.
- **A worked 3-way vig-removal example is embedded directly in the prompt.** Given Argentina -155 / Draw +265 / Ecuador +420, the vig-removed true probabilities come out to roughly Argentina 52%, Draw 24%, Ecuador 17%. Draw-No-Bet true probability for Argentina is then derived as $52\% \div (52\% + 17\%) \approx 75\%$. If the *posted* DNB line is -120 (implied 54.5%), that's a large exploitable edge; if the posted DNB line is -240, it blows through the -150 juice ceiling and is an automatic pass — never even a lean.
- **DNB prices must come from the live market feed — never be estimated from the 3-way moneyline.** This rule is stated in the prompt and then *independently re-enforced in code*: `esm/odds_client.py` fetches posted `draw_no_bet` lines as a dedicated API call, and `esm/soccer_odds.py::validate_wc_official_plays()` runs as a post-processing gate on Claude's own output, dropping any DNB play that lacks a genuinely posted line or that exceeds -150, and overwriting Claude-stated totals odds with the actual posted price. Any plays removed this way are logged into the card's `pass_notes` rather than silently disappearing — the system does not trust the model to police this rule perfectly on its own.
- **Situational filters** cover tournament openers, must-win/elimination games, host-nation advantage (USA/Canada/Mexico 2026), and altitude venues (Mexico City ~2,240m, Guadalajara ~1,590m, Denver ~1,609m). **Dead-rubber games are an automatic pass** — "*Unknown rotation, unmotivated sides — no edge model applies*" — including final group-stage games where either side has a bracket-dodging incentive to avoid winning.
- **Hard avoids, no exceptions:** first goalscorer, correct score, halftime/fulltime doubles, and props without confirmed lineups.

Data integrity and source hierarchy (Section 2)

The prompt defines a four-tier reliability hierarchy for input data: **Tier 1** — official injury reports, lineups, and the odds actually supplied in the prompt ("*treat supplied odds data as Tier 1*"); **Tier 2** — trusted projections/simulations; **Tier 3** — beat-reporter context; **Tier 4** — public chatter/social media, to be used "*last — as trap detector only*." Confirmed availability always beats a projection; current pricing always beats stale data. It also defines "consensus-trap" filters that penalize a play if it's repeated identically across multiple content sources (a sign of copy-paste groupthink rather than independent edge), and instructs the model to never invent numbers: any odds/lines/stats not present in the supplied data must be flagged in `pass_notes`, not fabricated.

Sharp vs. public interpretation

Both the system prompt and a dedicated companion module, `esm/split_guidance.py` (whose text is injected alongside the live market-intelligence data computed by `esm/market_intelligence.py`), teach the model how to read Action-Network-style betting splits: ticket % (number of bets) diverging sharply from money % (dollar handle) on the same side signals "sharp" or "public-heavy" action; a **reverse line move** — where the price improves for bettors despite heavy public money on that side — is a strong signal the book/sharp money is leaning the other way. The model is repeatedly told that splits are *confirmation*, never a primary edge source, and that decision order is always **fundamentals** → **market intelligence** → **price**.

Output contract

Claude must return *only* JSON (no prose, no markdown) matching a fixed schema, enforced by convention in the prompt and then structurally validated/enriched downstream in code:

```
{
  "date", "slate_grade" (A–F), "slate_grade_note",
  "official_plays": [ {
    id, sport, game, game_time_mdt, bet, market, book, odds,
    implied_prob_pct, true_prob_pct, edge_gap_pct, units,
    confidence (HIGH | MEDIUM | LEAN | FLYER),
    edge_summary, key_factors[], main_risk, last_playable_number,
    correlation_note, line_movement, sharp_action,
    market_signals: { flags[], steam_side, reverse_line,
                      public_bet_pct_home, public_money_pct_home,
                      sharp_money_flag, big_money_flag, data_quality }
  } ],
  "leans": [...], "quick_reads": [...], "pass_notes": [...],
  "running_record": { provided: false, summary }
}
```

`official_plays` is hard-capped at 5; a D/F slate grade permits an empty `official_plays` list; `correlation_note` is mandatory whenever two or more plays share a game, player, or narrative assumption. The `market_signals` and `line_movement` / `sharp_action` fields are actually *populated after the fact* by `esm/market_intelligence.py::enrich_card_with_market_signals()` — the model is asked to leave room for this data in its schema, but the authoritative values are computed deterministically from the odds/splits history the code already has, not trusted purely from the model's own claims.

Confidence tiers and voice

Four confidence tiers gate how a play is framed and (loosely) how much conviction it should carry: **HIGH** (edge clears threshold, role secure, 3+ aligned factors), **MEDIUM** (real edge, one lingering risk), **LEAN** (playable but thin), **FLYER** (justified only by price/parlay structure, not raw edge). A brand-voice section (Section 3) mandates a "sharp, modern, professional, peer-to-peer" tone and explicitly bans tout-speak — *"Hammer this."*, *"Free money."*, *"Lock of the century."*, *"Cannot lose."* are all forbidden phrasings, along with excessive em-dash usage.

Prompt caching – why the model is cheap to run continuously

Because `ESM_SYSTEM_PROMPT` is large (~550 lines) and byte-identical across every call — every agent scan for every user, every MLB card, every World Cup card — every call site wraps it as an Anthropic content block with `cache_control: {"type": "ephemeral"}` rather than as a plain string:

```
system=[{"type": "text", "text": ESM_SYSTEM_PROMPT,  
        "cache_control": {"type": "ephemeral"}}]
```

This appears identically in `agent/kernel.py` (agent scans — where the *dynamically personalized* prompt built by `agent/prompt.py`, which itself starts from `ESM_SYSTEM_PROMPT`, is cached), `services/agent_runner.py` (MLB/ESM cards), and `services/world_cup_card.py` (World Cup cards). Anthropic's prompt caching means the first call in a cache window pays the pricier cache-write rate (`$3.75 /MTok`, vs. plain input at `$3 /MTok`), but every subsequent call within the cache TTL pays only the cache-read rate (`$0.30 /MTok`) — a 10x discount — for that entire ~550-line block. Given that agent scans alone can run every 30–180 minutes across every active user, this is explicitly the platform's largest recurring Claude cost line item (per `esm/api_budget.py`'s own framing), and the architecture is deliberately shaped around exploiting the cache rather than treating it as an incidental optimization.

`esm/claude_config.py::estimate_call_cost_usd()` reads `cache_read_input_tokens` / `cache_creation_input_tokens` straight off each response's `usage` object to log accurate per-call cost given this setup, and `AGENT_SCANS_ENABLED` is a first-class kill switch specifically so the (expensive, continuous) agent-scan Claude calls can be disabled independently of the (cheap, once-daily) card pipelines while the product proves out profitability.

Model and token configuration

`esm/claude_config.py` centralizes the model choice (`CLAUDE_MODEL` env var, default `claude-sonnet-4-6`) and per-call-type output token caps: `CARD_MAX_TOKENS = 8192` for the richer daily cards, `AGENT_SCAN_MAX_TOKENS = 4096` for the more frequent, terser agent-scan calls.

Chapter 4: Per-User Agent Architecture — backend/agent/

This package is the "per-user agent" layer: the stateful entity that gets provisioned per signup, tracks its own bankroll/exposure, remembers what it has done, and runs the scan loop described in Chapter 2. All bankroll/sizing math here is **not** literal Kelly-criterion staking — it is a simple, transparent **percentage-of-current-bankroll fixed-unit model** (default 3% per unit, 6% max daily exposure), which the Kelly-tier language in the system prompt (Chapter 3) then scales in discrete multiples of.

backend/agent/__init__.py

Purpose: Package marker/docstring for the `agent` module, which implements the per-user "AgentEdge" agentic betting AI.

Key exports: None — the file contains only a one-line module docstring (`# AgentEdge – per-user agentic sports betting AI`).

How it works: Empty package init; establishes the module's identity/comment header only, no runtime logic.

Interacts with: Nothing directly; all other files in this report live under this package (`agent.*`).

backend/agent/bankroll.py

Purpose: Pure-math utilities for auto-calculated unit sizing and bankroll bookkeeping — the foundation of the platform's stake-sizing model.

Key exports:

- `DEFAULT_UNIT_PCT = 0.03` — default fraction of bankroll that equals 1 unit (3%).
- `DEFAULT_MAX_DAILY_PCT = 0.06` — default max fraction of bankroll exposed per day (6%, ~2 units).
- `compute_unit_size(bankroll, unit_pct=DEFAULT_UNIT_PCT) -> float` — dollar value of one unit, floored at \$1.
- `compute_max_daily_units(bankroll, unit_pct, max_daily_pct) -> int` — integer max units allowed per day given current bankroll.
- `compute_bankroll_summary(bankroll_current, bankroll_starting, unit_pct, max_daily_pct, units_at_risk) -> dict` — full bankroll/exposure/P&L snapshot dict used throughout the UI/prompt.
- `apply_bet_result(bankroll_current, units_result, unit_size) -> float` — applies a graded bet's unit result (+/-) to bankroll in dollars, floored at 0.

How it works: This is not Kelly-criterion sizing — it's a simple percentage-of-bankroll fixed-unit model.

`compute_unit_size` multiplies bankroll by `unit_pct` (default 3%) and rounds to 2 decimals, with a \$1 floor to avoid degenerate sizing on tiny bankrolls. `compute_max_daily_units` derives a *daily cap* by dividing the max-dollar exposure (`bankroll * max_daily_pct`) by the unit size, floored to at least 1 unit — meaning the daily unit cap is always recomputed off the *current* (dynamic) bankroll and unit size rather than being a fixed number of units.

`compute_bankroll_summary` composes these two functions plus P&L (`current - starting`) and P&L percentage into the dict the frontend/prompt consumes, tracking `units_remaining_today` as `max_units - units_at_risk` (never negative). `apply_bet_result` converts a bet's unit result into dollars via the unit size and adds it to bankroll, clamped at a floor of 0 to prevent negative bankroll.

Interacts with: Used by `bankroll_backfill.py`, `post_grade.py`, `provision.py`, `unit_tracker.py`, and indirectly `prompt.py` / `kernel.py` (via `get_unit_context`). Writes ultimately land in the `agent_instances` table.

backend/agent/bankroll_backfill.py

Purpose: Rebuilds/repairs `agent_instances.bankroll_current` by replaying a user's full graded bet history from the starting bankroll — a reconciliation/migration tool.

Key exports:

- `replay_bankroll_for_user(db, user_id) -> dict` — replays one user's graded bets chronologically to recompute `bankroll_current`.
- `backfill_all_agent_bankrolls(db) -> dict` — runs the replay for every user with an `agent_instances` row, aggregating results/errors.

How it works: `replay_bankroll_for_user` loads the user's `agent_instances` row (bailing out with `skipped` if none exists), reads `unit_pct` (default 3%) and starting bankroll, then queries all non-pending `bets` ordered by `date` then `created_at` (chronological replay order). For each bet it computes `units_result` via `normalize_units_result` (from `services.units`), recomputes the *current* unit size from the running bankroll (so unit size compounds/shrinks as bankroll changes bet-by-bet), and applies the result via `apply_bet_result`. After the loop it writes the final `bankroll_current` back to `agent_instances` and calls `refresh_stored_unit_size` (from `unit_tracker.py`) to sync `preferences.unit_size`. It returns a dict with starting/current bankroll, P&L, and count of bets replayed. `backfill_all_agent_bankrolls` iterates all `agent_instances.user_id` rows, catches per-user exceptions so one bad user doesn't abort the whole run, and returns aggregate counts plus per-user results.

Interacts with: `agent.bankroll` (unit-size/apply-bet math), `services.units.normalize_units_result`, `agent.unit_tracker.refresh_stored_unit_size`; reads/writes `agent_instances` and `bets` tables. Likely invoked as an admin/maintenance script or migration endpoint.

backend/agent/kernel.py

Purpose: The core per-user "agent scan loop" — the orchestration engine that gathers market/injury data, calls Claude to reason about it, and persists the resulting observations, hypotheses, beliefs, and bet positions.

Key exports:

- `run_agent_scan(db, user_id, trigger_type="scheduled_scan") -> dict` — main entry point; runs one full scan cycle for a user and returns a result summary.
- `_build_filtered_snapshot(db, today, sport_keys) -> dict` — internal: loads market odds snapshot (cache-first, live-API fallback) filtered to the user's sports.
- `_build_espn_context(market_snapshot, today) -> dict` — internal: fetches ESPN scoreboard/injury context per sport in the snapshot.
- `_build_scan_message(...) -> str` — internal: assembles the user-turn prompt text (market data, injuries, hypotheses, recent activity, instructions).
- `_summarize_market(snapshot) -> str` / `_summarize_espn(espn_context) -> str` — internal: human-readable text summarizers for prompt injection.
- `_call_agent(system_prompt, user_message) -> dict` — internal: calls the Anthropic API and parses JSON (stripping markdown fences) from the response.
- `_persist_positions(db, user_id, today, positions, max_plays, units_remaining) -> float` — internal: writes accepted bet positions to `cards` / `bets`, logs episodes, syncs exposure, triggers sheet sync; returns units used.

How it works: `run_agent_scan` is the scan loop: (1) load `agent_instances` row and bail if not `status == "active"`; (2) load `preferences` and `profiles` for personalization; (3) compute "today" in the configured timezone (`TIMEZONE` env, default `America/Denver`); (4) resolve the user's sport preferences into odds-API sport keys via `agent.sports`, restricted to major-league sports only (World Cup is excluded and handled by a separate pipeline) — if

none, log an observation episode and skip; (5) expire stale hypotheses and sync units-at-risk/bankroll context via `unit_tracker`; (6) build a filtered market snapshot (cache-first via `esm.snapshot_cache`, falling back to live `OddsClient` calls) — if zero games are on the slate, skip the Claude call entirely to save API cost, logging a "Quiet slate" observation instead; (7) build ESPN injury/scoreboard context via `StatsClient`; (8) gather beliefs, performance history (`learning.memory.get_performance_context`), watched hypotheses, and recent episode feed; (9) build the system prompt (`build_agent_system_prompt`) and user message, then call `_call_agent`, which hits the Anthropic API (model/token limits from `esm.claude_config`) with prompt caching on the system block (`cache_control: ephemeral`) and parses the JSON response (stripping ``json fences if present); (10) on success, update `agent_instances.mode` / timestamps, then fan out the structured response into side effects: log episodes for mode changes/observations/hypotheses/pass_notes, create hypothesis rows, upsert belief updates, and persist any recommended positions (capped by `max_plays` and remaining daily units) via `_persist_positions`, which writes to the `cards` table (backward-compatible dashboard) and inserts rows into `bets` (tagged `AGENT_BET_TAG = "agent"`), logging a "position" episode per accepted play, then re-syncs `units-at-risk` and triggers `maybe_sync_sheets`. Errors from Claude (API error or JSON parse failure) are logged as an "error" episode and the scan returns `{"error": "claude_failed"}` without crashing. ****Interacts with:**** `agent.unit_tracker` (unit context/sync, sport-key filtering), `agent.memory_store` (episodes, beliefs, hypotheses, prompt formatting), `agent.prompt.build_agent_system_prompt`, `agent.sports` (sport resolution/display), `esm.claude_config` (model name, token limits, usage logging), `esm.odds_client.OddsClient` and `esm.snapshot_cache` (market data), `esm.stats_client.StatsClient` (ESPN injuries/scoreboard), `learning.memory.get_performance_context`, `services.sheets_sync.maybe_sync_sheets`; reads/writes `agent_instances`, `preferences`, `profiles`, `cards`, `bets` tables; calls the Anthropic API directly.

backend/agent/memory_store.py

Purpose: CRUD and formatting layer for the agent's persistent memory — episodic feed, learned beliefs, and tracked hypotheses — that the scan loop reads/writes and injects into prompts.

Key exports:

- `get_agent_instance(db, user_id) -> dict | None` — fetch the `agent_instances` row.
- `log_episode(db, user_id, *, trigger_type, trigger_payload, episode_type, title, reasoning, action_payload) -> dict | None` — insert a row into `agent_episodes`.
- `get_feed(db, user_id, limit=50) -> list` — recent episodes, newest first.
- `get_beliefs(db, user_id, limit=20) -> list` — beliefs ordered by `last_validated` desc.
- `upsert_belief(db, user_id, category, belief, confidence=0.5) -> None` — insert new belief or bump confidence/evidence_count/last_validated on an existing identical belief text.
- `get_hypotheses(db, user_id, status="watching") -> list` — hypotheses filtered by status.
- `create_hypothesis(db, user_id, data) -> dict | None` — insert a hypothesis with a 24-hour `expires_at`.
- `format_beliefs_for_prompt(beliefs) -> str` — renders top-10 beliefs with % confidence for the system prompt.
- `expire_stale_hypotheses(db, user_id=None) -> int` — marks `watching` hypotheses past `expires_at` as expired; returns count updated.
- `link_position_episode_outcome(db, bet, result, units_result, *, tag="") -> bool` — attaches W/L/P outcome + lesson text to the matching "position" episode for a graded bet.
- `_find_position_episode(db, user_id, bet) -> str | None` — internal matcher (by `bet_id` in `action_payload`, then exact bet text match, then title match) to locate the episode tied to a bet.
- `_grade_lesson(result, units_result, tag) -> str` — internal: builds a short lesson string ("Win (+1.5u). ..." /

"Loss (-2.0u). Review thesis...").

- `format_hypotheses_for_prompt(hypotheses) -> str` — renders up to 8 watched hypotheses for the scan prompt.
- `format_recent_episodes_for_prompt(episodes) -> str` — renders up to 12 filtered recent episodes (with outcome/lesson) for prompt continuity.

How it works: All functions are thin wrappers around Supabase-style `db.table(...)` query-builder calls (select/insert/update/eq/order/limit/in/_lt/is_), with no ORM abstraction beyond this module — it is the data-access layer for three tables (`agent_episodes`, `agent_beliefs`, `agent_hypotheses`) plus a read on `agent_instances`. `upsert_belief` implements dedupe-by-exact-text: if a belief with identical text already exists for the user, it bumps `evidence_count` and refreshes `confidence` / `last_validated` rather than inserting a duplicate, giving beliefs a lightweight reinforcement mechanic. `create_hypothesis` always sets a 24-hour TTL (`expires_at`), and `expire_stale_hypotheses` is the sweep that flips expired `watching` rows to `expired` status (batched via `.in("id", ids)`). The episode-outcome linking (`link_position_episode_outcome` / `_find_position_episode`) is the bridge between bet grading and the agent's episodic memory: it searches the last 25 un-graded "position" episodes for the current user, matching first by the `bet_id` embedded in `action_payload`, falling back to literal bet-text equality, then title equality — a heuristic three-tier match since episodes aren't foreign-keyed to `bets` rows directly. The `format_*_for_prompt` functions are pure string builders that truncate/limit content (beliefs to top 10, hypotheses to top 8, episodes to top 12 of specific types, reasoning snippets to 200 chars) to keep prompt size bounded.

Interacts with: Called heavily by `kernel.py` (episode/belief/hypothesis writes and prompt formatting) and by `post_grade.py` (`expire_stale_hypotheses`, `link_position_episode_outcome`) and `provision.py` (`log_episode`, `upsert_belief`). Operates on Supabase tables: `agent_instances`, `agent_episodes`, `agent_beliefs`, `agent_hypotheses`.

backend/agent/post_grade.py

Purpose: Side-effect handlers that run after a bet is graded (W/L/P) — updating the user's bankroll and closing the loop on the agent's episodic memory for learning.

Key exports:

- `update_bankroll_after_grade(db, bet, units_result) -> bool` — applies a graded bet's P&L to `agent_instances.bankroll_current`.
- `apply_post_grade_effects(db, bet, result, units_result, *, tag="") -> None` — runs all per-bet post-grade hooks (bankroll update + episode outcome linking).
- `finalize_grade_batch(db, affected_user_ids: set[str]) -> None` — batch cleanup (expire stale hypotheses) for all users affected by a grading run.

How it works: `update_bankroll_after_grade` looks up the user's `agent_instances` row (returns `False` if missing/no `user_id`), reads `unit_pct` (default 3%), computes the *current* unit size off the current bankroll, applies the bet's `units_result` via `apply_bet_result`, writes the new `bankroll_current`, and then calls `refresh_stored_unit_size` (from `unit_tracker.py`) so `preferences.unit_size` stays in sync with the newly updated bankroll — meaning unit size auto-shrinks/grows after every graded bet, not just periodically. `apply_post_grade_effects` is the single call site meant to be invoked by the bet-grading pipeline per graded bet: it chains the bankroll update with `link_position_episode_outcome` (from `memory_store.py`) to tag the corresponding "position" episode with its outcome and a lesson string. `finalize_grade_batch` is meant to run once after a batch grading job processes many bets/users, sweeping `expire_stale_hypotheses` per affected user so grading (which often coincides with the start of a new period) also triggers hypothesis hygiene.

Interacts with: `agent.bankroll` (unit size/apply math), `agent.memory_store` (`expire_stale_hypotheses`,

`link_position_episode_outcome`), `agent.unit_tracker.refresh_stored_unit_size`; reads/writes `agent_instances`. Called by `services/grader.py` once results are known.

backend/agent/prompt.py

Purpose: Builds the full Claude system prompt for a specific user's AgentEdge instance, combining the shared ESM system prompt with per-user identity, bankroll, constraints, and the required JSON output schema.

Key exports:

- `build_agent_system_prompt(*, user_name, bankroll_current, bankroll_starting, unit_size, max_daily_units, units_at_risk, risk_level, sports, bet_types, include_parlays, mode, beliefs_text, performance_text) -> str` — the sole function; returns the concatenated system prompt string.

How it works: Computes `units_remaining = max(0, max_daily_units - units_at_risk)` and P&L (`bankroll_current - bankroll_starting`) with a sign prefix for display. It then builds an `identity` block (large formatted text) declaring the agent's identity ("AgentEdge by EdgeSportsMedia"), current mode, bankroll figures, units at risk/remaining, user constraints (sports, bet types, risk level, parlay enable/disable), agentic behavior rules (default to observe/track, never exceed daily exposure, never chase losses, only recommend enabled sports/bet types, size using the auto-calculated unit), and injects the caller-supplied `beliefs_text` and `performance_text` (memory hierarchy: user's own history trusted over "platform agent brain" collective patterns). Separately it defines a fixed `output_schema` block that mandates the agent return *only* JSON matching a specific schema: `mode`, `mode_reason`, `observations[]`, `hypotheses[]`, `positions[]`, `pass_notes[]`, `belief_updates[]`, with inline rules (always ≥ 1 observation even on quiet slates, positions only when edge is clear and within exposure limits, `pass_notes` to demonstrate selectivity, use `edge_summary` not `why`). The final return value is `ESM_SYSTEM_PROMPT + identity + output_schema` — i.e., the platform-wide base prompt is always prepended before per-user personalization.

Interacts with: `esm.system_prompt.ESM_SYSTEM_PROMPT` (shared base prompt). Called exclusively by `kernel.py` (`run_agent_scan`) to build the `system` block for the Anthropic API call; the schema it documents corresponds exactly to the JSON `kernel.py._call_agent` parses back out.

backend/agent/provision.py

Purpose: Provisions (creates/updates) a user's AgentEdge agent instance and preferences during onboarding, and reports current agent status for the UI.

Key exports:

- `provision_agent(db, user_id, setup: dict) -> dict` — creates/upserts `agent_instances` and `preferences` rows from onboarding setup data, seeds initial beliefs, logs an "agent online" episode.
- `get_agent_status(db, user_id, prefs=None) -> dict` — returns provisioning/status/mode/bankroll-summary/preferences for a user, or `{"provisioned": False, "status": "pending_setup"}` if no instance exists.

How it works: `provision_agent` pulls onboarding fields out of `setup` with sensible defaults (bankroll \$1000, `unit_pct` 3%, `max_daily_pct` 6%, sports `["MLB", "WC"]`, bet types `["player_props", "straight"]`, risk `MEDIUM`, `max_plays` 5, parlays disabled). It computes the initial bankroll summary via `compute_bankroll_summary` (`bankroll_current == bankroll_starting` at provisioning time) to derive `unit_size / max_units`. It then `upsert`s (on `user_id` conflict) an `agent_instances` row with `status="active"`, `mode="scanning"`, and the bankroll/unit config, and a parallel `preferences` upsert with the user's sport/bet-type/risk/parlay selections plus the computed `unit_size`. It seeds four initial `agent_beliefs` rows via `upsert_belief` (profile risk/max-plays, bankroll/unit-size explanation, sport scope, allowed bet types/parlay rule) all at `confidence=1.0` since they're user-declared facts, not learned ones. Finally it logs an "Agent online" system episode summarizing bankroll/unit size/sports and returns `{"provisioned":`

`True, "agent": agent_row, "bankroll": summary}`. `get_agent_status` is a read-only status endpoint: it loads `agent_instances` (returning early if none), optionally loads `preferences` if not passed in, recomputes a live bankroll summary via `compute_bankroll_summary` (using stored `units_at_risk`), and returns a status dict combining provisioning state, mode, bankroll, timestamps, subscription tier, and a trimmed preferences view.

Interacts with: `agent.bankroll` (bankroll summary/unit math), `agent.unit_tracker` (`DEFAULT_UNIT_PCT`, `DEFAULT_MAX_DAILY_PCT` constants), `agent.memory_store` (`log_episode`, `upsert_belief`); writes `agent_instances` and `preferences` tables. Called by the `/api/agent/setup` onboarding route and the `/api/agent` status route in `main.py`.

backend/agent/sports.py

Purpose: Central registry mapping user-facing sport labels (e.g. "MLB", "WC") to internal odds-API sport keys (e.g. "baseball_mlb"), and helpers to resolve/display/filter sports for polling and agent scans.

Key exports:

- `SPORT_REGISTRY: dict[str, dict]` — maps labels (MLB, NBA, NHL, NFL, WC, NCAAB, NCAAF) to `{key, label, display, season_active}` entries.
- `MAJOR_SPORTS = ["MLB", "NBA", "NHL", "NFL", "WC"]` — the "major league" label set used to scope agent behavior.
- `user_sport_to_key(sport_label) -> str | None` — resolves a user preference label to an odds-API key, falling back to passing through a raw API key if it's already in `ACTIVE_SPORTS`.
- `resolve_user_sports(user_sports: list[str]) -> list[str]` — maps a list of user sport labels to a deduplicated list of odds-API keys.
- `get_active_sports_for_polling() -> list[str]` — returns odds-API keys for sports currently in-season (`season_active=True`) and present in `ACTIVE_SPORTS`.
- `get_all_supported_sports() -> list[dict]` — full list of selectable sports (major + NCAAB/NCAAF) with `has_props` flag derived from `PROP_MARKETS`.
- `sport_key_to_display(sport_key) -> str` — reverse lookup from odds-API key to display label, falling back to a title-cased reformat of the key.

How it works: `SPORT_REGISTRY` is a static dict hardcoding 7 sports; currently only MLB and WC ("World Cup Soccer") have `season_active: True`, reflecting real-world season state as of this snapshot. `user_sport_to_key` normalizes the label to uppercase and looks it up in the registry; if not found, it checks whether the raw label is already a valid `ACTIVE_SPORTS` key (from `esm.config`) and passes it through, otherwise returns `None`. `resolve_user_sports` deduplicates while preserving order. `get_active_sports_for_polling` is the "in-season" filter used by the shared background poller that refreshes odds data. `get_all_supported_sports` filters the registry down to `MAJOR_SPORTS` plus NCAAB/NCAAF and annotates each with whether player props exist for it. `sport_key_to_display` does a linear scan over the registry values to reverse-map an API key back to a short display code.

Interacts with: `esm.config.ACTIVE_SPORTS` and `esm.config.PROP_MARKETS`. Consumed by `kernel.py` (`resolve_user_sports`, `sport_key_to_display`) and `unit_tracker.py` (`major_league_sport_keys` filters out `WC_SPORT_KEY`, which matches this file's `WC` entry's key `soccer_fifa_world_cup`).

backend/agent/unit_tracker.py

Purpose: Tracks and recalculates a user's daily unit exposure ("units at risk") from pending bets, computes live bankroll/unit context, and keeps `preferences.unit_size` in sync — while explicitly carving out World Cup bets from the major-league daily exposure cap.

Key exports:

- `TIMEZONE` (env `TIMEZONE`, default `America/Denver`), `DEFAULT_UNIT_PCT` (env `UNIT_PCT`, default 0.03), `DEFAULT_MAX_DAILY_PCT` (env `MAX_DAILY_PCT`, default 0.06) — module-level config constants, overridable by environment variables.
- `WC_BET_TAG = "world_cup"`, `ESM_BET_TAG = "esm"`, `AGENT_BET_TAG = "agent"` — bet tagging constants for `post_slate_tag`.
- `WC_SPORT_KEY = "soccer_fifa_world_cup"` — World Cup odds-API sport key constant.
- `today_mt() -> str` — today's date string in the configured timezone.
- `is_wc_bet(bet: dict) -> bool` — determines if a bet is a World Cup bet via `post_slate_tag == "world_cup"` or a legacy `"[WC]"` prefix in `notes`.
- `sum_pending_units(db, user_id, bet_date, *, exclude_wc=True) -> float` — sums `units` of pending bets for a date, optionally excluding WC bets.
- `sum_wc_pending_units(db, user_id, bet_date) -> float` — sums pending WC-only bet units for a date.
- `sync_units_at_risk(db, user_id, bet_date=None) -> dict` — recomputes and persists `agent_instances.units_at_risk` from pending major-league bets; returns both major-league and WC pending units.
- `get_unit_context(db, user_id, bet_date=None) -> dict` — full bankroll+exposure summary with live unit size, falling back to `preferences` if no `agent_instances` row exists yet.
- `refresh_stored_unit_size(db, user_id) -> float` — recomputes unit size from current bankroll and writes it to `preferences.unit_size`.
- `major_league_sport_keys(sport_keys: list[str]) -> list[str]` — filters out the World Cup sport key from a list of sport keys.

How it works: This module is the exposure-accounting layer sitting on top of `bankroll.py`'s pure math. `is_wc_bet` treats a bet as World Cup either via explicit tag or a legacy notes-prefix convention, which

`sum_pending_units` / `sum_wc_pending_units` both use to partition pending bets by date into "major league" vs "WC" buckets — WC bets are deliberately excluded from the daily major-league cap because World Cup is handled by the separate daily WC card. `sync_units_at_risk` is the write path: it queries pending bets for the date, sums non-WC units, and if an `agent_instances` row exists, persists that sum into `units_at_risk` — this is called at the start of every scan (`kernel.py`) and after any position is written, so `units_at_risk` is always recomputed from source-of-truth pending bets rather than incrementally tracked. `get_unit_context` is the read path used to build prompts: it first calls `sync_units_at_risk` to refresh exposure, then loads `agent_instances` for bankroll/unit_pct/max_daily_pct — with a fallback branch (used pre-provisioning) that instead reads `preferences.bankroll_starting` as a stand-in for both current and starting bankroll — then calls `compute_bankroll_summary` (from `bankroll.py`) to produce the full summary dict, additionally tacking on `wc_units_at_risk` and `unit_pct`. `refresh_stored_unit_size` recomputes context and pushes the resulting `unit_size` into `preferences.unit_size` with an updated timestamp — called after any bankroll-changing event (`post_grade.py`, `bankroll_backfill.py`) to keep the displayed/preferences unit size current. `major_league_sport_keys` is a simple filter used by `kernel.py` to strip WC out of the sports the agent scan loop operates on.

Interacts with: `agent.bankroll.compute_bankroll_summary` / `compute_unit_size`; reads/writes `agent_instances`, reads `bets`, reads/writes `preferences`. Called extensively by `kernel.py` (scan-start sync, unit context for prompt, post-position sync), `post_grade.py` and `bankroll_backfill.py` (`refresh_stored_unit_size`), and `provision.py` (imports `DEFAULT_UNIT_PCT` / `DEFAULT_MAX_DAILY_PCT` / `sync_units_at_risk`).

Chapter 5: Data & Intelligence Layer — `backend/esm/`

This package — "ESM" for Edge Sports Media — is where the platform's external world touches the model: odds providers, injury/stats data, betting-splits scraping, market-intelligence analytics (line movement, steam detection, sharp-vs-public interpretation), Claude API configuration/cost tracking, and the World Cup soccer-specific data-integrity guardrails.

`esm/system_prompt.py`, the single most important file in this module, is covered in full in Chapter 3 and is not repeated here.

`backend/esm/__init__.py`

Purpose: Package marker file for the `esm` (Edge Sports Media) module.

Key exports: None — the file is completely empty (0 bytes).

How it works: Exists solely so Python treats `esm/` as an importable package; contains no code, docstring, or `__all__` declarations.

Interacts with: Enables `from esm.xxx import ...` imports used throughout `backend/agent/` and `backend/services/`.

`backend/esm/system_prompt.py`

Purpose: Defines `ESM_SYSTEM_PROMPT` — the complete ~550-line "ESM Framework" system prompt sent to Claude on every card-generation and agent-scan call. This is the single most important file in the platform, since it is the model's persona, betting philosophy, market-specific rules, and required JSON output contract.

Full analysis: See **Chapter 3: How the AI Model Is Designed**, which covers this file in complete detail (persona/mandate, Kelly-tier sizing rules, World Cup 3-way vig-removal math, data-integrity hierarchy, output schema, and the prompt-caching strategy applied to it) rather than repeating it here.

Interacts with: Consumed verbatim (or as part of a larger composed prompt in `agent/kernel.py` / `agent/prompt.py`) by `services/agent_runner.py`, `services/world_cup_card.py`, and `agent/kernel.py`. Its output JSON schema is the contract that `esm/soccer_odds.py::validate_wc_official_plays()` and `esm/market_intelligence.py::enrich_card_with_market_signals()` both operate on after Claude responds.

`backend/esm/action_network_client.py`

Purpose: Scrapes Action Network's public "public-betting" pages (no API key needed) to extract ticket % (public bets) and money % (handle) splits per market side, used as the "sharp vs. public" signal layer.

Key exports:

- `BASE_URL`, `DK_BOOK_ID` ("15"), `USER_AGENT`, `SPORT_TO_AN_PATH`, `TEAM_ALIASES` — constants/config for mapping internal sport keys to Action Network URL paths and normalizing team names.
- `fetch_public_splits(sport_key)` — fetches and parses all games' splits for a sport from the AN public-betting page.
- `all_mapped_sport_keys()` — lists sport keys that have an AN mapping.
- `match_event_to_game(an_event, away_team, home_team)` — fuzzy-matches an AN event to an odds-snapshot game by team name.
- Internal helpers: `_normalize_team`, `_fetch_next_data`, `_classify_sharp`, `_split_row`, `_extract_dk_event_markets`.

How it works: `_fetch_next_data` GETs the AN page and regex-extracts the embedded `__NEXT_DATA__` Next.js JSON

blob (AN does not expose a documented public API). `_extract_dk_event_markets` reads DraftKings (book id "15") market data per game, pulling `bet_info.tickets.percent` and `bet_info.money.percent` for moneyline, spread, and total markets, keyed into `h2h_home / h2h_away / h2h_draw`, `spread_home / spread_away`, `total_over / total_under`. `_classify_sharp` derives a `sharp / big_money / public_heavy` label purely from the ticket% vs money% divergence (≥ 15 pt gap $\rightarrow$ "sharp"; ≥ 10 pt gap with money% $\geq 55 \rightarrow$ "big_money"; ticket% ≥ 70 with money% 10pt+ lower $\rightarrow$ "public_heavy"). `fetch_public_splits` filters World Cup games to `league_name == "worldcup"` since the soccer page mixes leagues.

Interacts with: Consumed by `services/splits_sync.py` which persists results into the `market_splits` table; results are read back by `esm/market_intelligence.py`.

backend/esm/api_budget.py

Purpose: Centralizes quota math and burn-rate estimation for the two odds providers (The Odds API "TOA" and SportsGameOdds "SGO"), deciding when it's safe to call each and how often to poll given free-tier limits.

Key exports:

- Constants: `TOA_MONTHLY_QUOTA` (20000), `TOA_RESERVE` (500), `TOA_MAINLINES_MIN`, `TOA_PROPS_MIN` (1000), `TOA_PROPS_PER_GAME_RESERVE`, `TOA_MAINLINE_MARKETS` (3), `TOA_PROP_BATCH_SIZE` (4), `SGO_MONTHLY_OBJECTS` (2500), `SGO_RESERVE` (200), `SGO_RATE_LIMIT_PER_MIN` (10), `SGO_MIN_REQUEST_INTERVAL_SEC`, `POLL_INTERVAL_MINUTES` (360), `AGENT_SCAN_INTERVAL_MINUTES` (180), `SNAPSHOT_MAX_AGE_MINUTES` (360), `ODDS_PRIMARY_SOURCE` ("sgo"), `TOA_MAX_CREDITS_PER_SNAPSHOT` (150) — all env-overridable.
- `ApiUsageState` (dataclass) — runtime-tracked remaining/used credits/objects from response headers, plus `warnings` list.
- `estimate_toa_snapshot_cost(...)`, `estimate_sgo_snapshot_cost(...)`, `estimate_daily_burn(...)` — cost projection functions.
- `should_use_toa`, `should_fetch_toa_props`, `should_use_sgo` — gating predicates that compare `ApiUsageState` against reserve thresholds.
- `budget_summary(usage)` — human-readable dashboard payload combining both providers' status.

How it works: TOA costs credits per market group (h2h+spreads+totals = 3 credits) plus 1 credit per prop-market batch per game; SGO costs "objects" = number of events returned (all markets bundled in one request). `_recommend_poll_interval` computes a poll cadence that keeps SGO usage under ~80% of its monthly quota over 30 days, clamped to [10, 360] minutes. `should_use_*` functions return `True` (optimistic) when usage is unknown (i.e., first ever call), otherwise enforce the reserve floor. `_status_label` classifies remaining quota as `critical / low / ok`.

Interacts with: Imported by `esm/odds_client.py` (gating decisions on every fetch) and `esm/config.PROP_MARKETS` (via lazy import in `_default_prop_markets` to avoid circular imports); surfaced to an admin dashboard via `budget_summary`.

backend/esm/claude_config.py

Purpose: Central configuration for Claude API model selection, token caps, and cost estimation/logging used across all Claude-calling services.

Key exports:

- `MODEL` (env `CLAUDE_MODEL`, default `"claude-sonnet-4-6"`).
- `CARD_MAX_TOKENS` (8192), `AGENT_SCAN_MAX_TOKENS` (4096) — output token caps per call type.
- `AGENT_SCANS_ENABLED` — feature flag to disable the (expensive) continuous agent-scan Claude calls, e.g. to control cost on Railway before profitability.
- `INPUT_COST_PER_MTOK` (3.0), `OUTPUT_COST_PER_MTOK` (15.0), `CACHE_READ_COST_PER_MTOK` (0.30),

`CACHE_WRITE_COST_PER_MTOK` (3.75) — Sonnet list pricing for logging only.

- `estimate_call_cost_usd(usage)` — computes approximate USD cost from an Anthropic `usage` object, splitting input tokens into cache-read vs plain and adding cache-write and output costs.
- `log_claude_usage(caller, usage, *, extra="")` — prints a one-line cost/token breakdown per Claude call.

How it works: `estimate_call_cost_usd` treats `cache_read_input_tokens` as billed at the cheap cache-read rate, subtracts them from `input_tokens` to get "plain" input tokens billed at full rate, and adds `cache_creation_input_tokens` at the (higher) cache-write rate — mirroring Anthropic's prompt-caching billing model. This is explicitly "not billing-grade," for logging/observability only.

Interacts with: Called by every Claude-invoking module (`agent/kernel.py` , `services/agent_runner.py` , `services/world_cup_card.py`) immediately after `client.messages.create(...)` to log spend; `MODEL / *_MAX_TOKENS` feed directly into those `messages.create` calls.

`backend/esm/config.py`

Purpose: Static configuration for which sports are actively tracked and which player-prop markets are requested per sport.

Key exports:

- `ODDS_API_KEY` , `SGO_API_KEY` — API keys from env.
- `ACTIVE_SPORTS` — list of 6 sport keys: `basketball_nba` , `baseball_mlb` , `icehockey_nhl` , `americanfootball_nfl` , `basketball_ncaab` , `soccer_fifa_world_cup` .
- `PROP_MARKETS` — dict mapping each sport key to a list of prop market identifiers (e.g. NBA: `player_points` , `player_rebounds` , ...; MLB: `batter_hits` , `pitcher_strikeouts` , ...; soccer: `player_goal_scorer_anytime` , `player_shots_on_target`).
- `DEFAULT_BOOK` = `"draftkings"` .

How it works: Pure declarative config, no logic — just the source of truth for sport/market coverage that both `odds_client.py` and `api_budget.py` iterate over.

Interacts with: Imported by `odds_client.py` (drives TOA/SGO snapshot construction), `api_budget.py` (`_default_prop_markets`), and mirrors the same sport-key set `stats_client.SPORT_MAP` uses.

`backend/esm/market_intelligence.py`

Purpose: Derives "market intelligence" signals — line movement/steam detection and public-vs-sharp betting split interpretation — that get merged into each game and surfaced to Claude as secondary confirmation evidence (never the primary edge driver).

Key exports:

- `STEAM_ODDS_THRESHOLD` (15 cents), `STEAM_TOTAL_THRESHOLD` (0.5 pts), `LOOKBACK_HOURS` (48) — env-tunable steam-detection thresholds.
- `analyze_game_lines(history, home_team, away_team, current_lines, external_splits)` — builds the per-game `intel` dict (flags, steam side, reverse-line flag, public/money percentages).
- `build_market_intelligence(db, snapshot, sport_keys)` — orchestrates per-game analysis across an entire snapshot, pulling DB history + splits for each game.
- `format_intelligence_for_prompt(intelligence)` — renders the intel dict tree into the human-readable text block injected into Claude's user message.
- `attach_intelligence_to_snapshot` , `sharp_action_label` , `line_movement_label` , `enrich_card_with_market_signals` — attach/format intel for downstream snapshot embedding, dashboard labels, and post-hoc enrichment of Claude's output card with `line_movement` / `sharp_action` / `market_signals` fields.
- `PROP_MARKETS_HINT` — frozenset of prop market keys used to detect "this play is a player prop" (no prop-level splits exist,

so only game-level context is attached).

- Internal: `_parse_ts`, `_american_implied`, `_odds_improved_for_bettors`, `_find_game_history`, `_load_external_splits`, `_apply_split_fields`, `_apply_external_splits`, `_game_line_context_note`, `_play_split_context`.

How it works: `_find_game_history` queries Supabase `market_snapshots` for a sport/event within a 48h lookback and reconstructs a time series of `lines` per capture. `analyze_game_lines` compares the *opening* snapshot (`history[0]`) against `current_lines`: a ≥ 15 -cent swing in either team's moneyline flags `steam_ml_{side}`; a ≥ 0.5 -point total move flags `steam_total`; juice moves ≥ 15 cents on over/under flag `steam_under_juice` / `steam_over_juice`. `_load_external_splits` first tries `services.splits_sync.find_splits_for_matchup` (name-matched, since Action Network event IDs don't align with TOA/SGO event IDs), falling back to a direct `market_splits` table query by `event_id`. `_apply_external_splits` layers in derived flags: `public_heavy_over/under` ($\geq 70\%$ one-sided ticket %), `contrarian_under/over_setup` (public $\geq 65\%$ one side, $\leq 35\%$ other), `sharp_money_under` (public on Over $\geq 60\%$ but money% favors Under by ≥ 10 pts), and **reverse line** detection — public $\geq 65\%$ on a side while that side's price *improved* for bettors (odds moved in bettors' favor despite the crowd being on it, meaning the book/sharp money pushed against the public). `enrich_card_with_market_signals` post-processes a finished Claude JSON card: it matches each `official_plays` / `leans` entry back to its game via `soccer_odds.parse_game_matchup` and attaches `line_movement`, `sharp_action`, and a full `market_signals` dict (including a play-specific `_play_split_context` mapping the bet text to the correct side's public/money %).

Interacts with: Reads `market_snapshots` and `market_splits` Supabase tables; calls `services.splits_sync.find_splits_for_matchup`; imports `esm.soccer_odds.parse_game_matchup`; its `format_intelligence_for_prompt` output is injected into the Claude user message (paired with `system_prompt.py`'s "MARKET INTELLIGENCE" instructions and `split_guidance.py`'s interpretation rules); `enrich_card_with_market_signals` populates the `market_signals` object defined in the system prompt's output JSON schema.

backend/esm/odds_client.py

Purpose: The unified odds-fetching client — abstracts over two providers (SportsGameOdds primary, The Odds API fallback) to build a normalized "market snapshot" (lines + player props per game per sport) while respecting budget constraints from `api_budget.py`.

Key exports:

- `OddsClient` class — `build_market_snapshot(target_date, sport_keys, force_source)` is the main entry point; class-level shared `_usage: ApiUsageState`; `get_usage()` classmethod.
- `SPORT_TO_SGO_LEAGUE`, `TOA_ONLY_SPORTS` (`soccer_fifa_world_cup`), `SGO_STAT_MAP` (per-sport SGO statID → internal market key mapping), `SGO_SPORT_TOKENS`, `ODDS_API_BASE`, `SGO_BASE`.
- Internal TOA implementation: `_toa_build_snapshot`, `_toa_get`, `_toa_day_window_utc`, `_toa_extract_best_lines`, `_toa_fetch_dnb_lines`, `_toa_extract_dnb_lines`, `_toa_extract_props`.
- Internal SGO implementation: `_sgo_build_snapshot`, `_sgo_get`, `_sgo_fetch_usage`, `_sgo_extract_lines`, `_sgo_extract_props`, `_sgo_best_book_price`, `_sgo_best_book_price_with_name`, `_sgo_entity_to_name`.
- Shared: `_merge_snapshots`, `_is_better_price`, `_is_soccer_sport`.

How it works: `OddsClient.build_market_snapshot` picks a strategy based on `force_source` or `ODDS_PRIMARY_SOURCE`: `_try_sgo_then_toa` (default) fetches SGO for all SGO-supported sports, then fills in World-Cup-only/missing sports via TOA, merging with `_merge_snapshots`; `_try_toa_then_sgo` is the reverse fallback chain. TOA calls (`_toa_get`) parse `x-requests-remaining` / `-used` / `-last` response headers into the shared `ApiUsageState`, treat HTTP 401/402/422 as quota exhaustion, and enforce a `TOA_MAX_CREDITS_PER_SNAPSHOT` (150)

safety cap mid-loop. `_toa_build_snapshot` fetches `/sports/{sport}/odds` for mainlines (h2h/spreads/totals) across DraftKings → FanDuel → BetMGM priority, conditionally fetches player props in batches of 4 markets per request (gated by `should_fetch_toa_props` /remaining credits), and for soccer additionally calls `_toa_fetch_dnb_lines` to get **posted draw_no_bet** lines (critical: soccer DNB odds are fetched live, never estimated). `_sgo_build_snapshot` calls SGO `/events/` per league/day window and parses SGO's compact `{statID}-{entityID}-{periodID}-{betTypeID}-{sideID}` odd-ID encoding via `_sgo_extract_lines` / `_sgo_extract_props`, resolving player names from entity IDs by stripping sport-league suffix tokens and trailing numeric IDs (`_sgo_entity_to_name`). Best price selection across books uses `_is_better_price` (higher payout wins, accounting for positive/negative American odds sign rules).

Interacts with: Imports `esm.config` (keys, `ACTIVE_SPORTS`, `PROP_MARKETS`, `DEFAULT_BOOK`) and `esm.api_budget` (gating/ `ApiUsageState`); output snapshot is stored via `esm.snapshot_cache.store_snapshot`, read back via `get_cached_snapshot`, enriched by `market_intelligence.py`, and validated for soccer plays by `soccer_odds.validate_wc_official_plays` (which relies on the `home_dnb` / `away_dnb` fields this module populates).

backend/esm/snapshot_cache.py

Purpose: Implements the shared-cache pattern so that odds are polled once (by a scheduled poller) and read many times by per-user agent scans, avoiding redundant API spend.

Key exports:

- `TIMEZONE` = "America/Denver".
- `get_cached_snapshot(db, sport_keys, target_date=None)` — reads the latest per-sport snapshot rows from Supabase, returns `None` if any/all are missing or older than `SNAPSHOT_MAX_AGE_MINUTES`.
- `store_snapshot(db, sport_key, sport_data)` — inserts a new snapshot row.
- `cache_age_minutes(db, sport_key)` — age of the latest cached row in minutes.

How it works: `get_cached_snapshot` loops each requested sport key, does a `market_snapshots` table query `order by captured_at desc limit 1`, parses ISO timestamps (defaulting naive timestamps to UTC), and skips sports whose latest capture is older than the cutoff (`SNAPSHOT_MAX_AGE_MINUTES`, default 360 = 6h from `api_budget.py`). It returns a combined `{date, sports, source: "cache", cached_at: <oldest capture timestamp>}` dict only if at least one sport has fresh data — sports with stale/missing cache are simply omitted rather than failing the whole call.

Interacts with: Reads/writes Supabase `market_snapshots` table; `store_snapshot` is called after `OddsClient.build_market_snapshot` succeeds (poller); `get_cached_snapshot` is called by per-user agent-scan code paths instead of hitting `OddsClient` directly; depends on `esm.api_budget.SNAPSHOT_MAX_AGE_MINUTES`.

backend/esm/soccer_odds.py

Purpose: Soccer/World-Cup-specific odds helpers enforcing data integrity for Draw No Bet (DNB) bets and the -150 juice ceiling, plus generic game-matchup string parsing shared across the ESM layer.

Key exports:

- `WC_JUICE_CEILING` = -150 (American odds).
- `is_dnb_bet(bet_text)`, `parse_dnb_team(bet_text)` — detect/extract DNB bet labels (e.g. "Draw No Bet — Germany").
- `parse_game_matchup(game_str)` — parses "Away vs Home" / "Away @ Home" / "Away at Home" / "Away v Home" strings (stripping trailing (Group X) annotations) into (away, home) tuples.
- `within_juice_ceiling(american_odds)` — True if odds ≥ -150.
- `find_game_in_snapshot(snapshot, game_str)` — fuzzy-matches a Claude-generated `game` label back to the raw odds snapshot game object.
- `posted_dnb_odds(game, team)` — returns the posted `home_dnb` / `away_dnb` price for a team from the snapshot.

- `validate_wc_official_plays(card, snapshot)` — post-processing validator/sanitizer run on Claude's output card for World Cup slates.

How it works: `validate_wc_official_plays` is a hard data-integrity gate: for every DNB play in `official_plays`, it looks up the matching game in the snapshot, requires a *posted* `draw_no_bet` price to exist (never inventing one — enforcing the system prompt's "Never estimate or invent DNB prices" rule), and drops the play if no posted line exists or if the posted price exceeds -150 juice. For totals bets it re-syncs `play["odds"]` to the actual posted `under_odds` / `over_odds` from the snapshot (overwriting whatever Claude stated) and updates `book`. Any remaining play whose odds still exceed -150 is also dropped. All removals are logged and appended to `card["pass_notes"]` so the discarded plays are visible in the final output rather than silently vanishing.

Interacts with: Called from `services/world_cup_card.py` after Claude returns its JSON, using the same snapshot `OddsClient` produced; `parse_game_matchup` is reused by `market_intelligence.enrich_card_with_market_signals` to match plays to games.

`backend/esm/split_guidance.py`

Purpose: A small shared text-snippet module holding the prompt language that teaches Claude how to interpret Action Network-style public/money betting splits.

Key exports: `SPLIT_INTERPRETATION_GUIDANCE` — a multi-paragraph string constant.

How it works: Pure string constant, no logic. It encodes the interpretive rules: decision order is fundamentals → market intel → price (never let splits create edge on their own); ticket % (public bets) vs money % (handle) divergence semantics (ticket>>money → fade-candidate/public-heavy; money>>ticket → sharp/confirmation signal); reverse-line respect; totals-splits are context-only (never pick Over/Under solely on ticket %); explicit note that player props have no prop-level split feed, so game-level splits are secondary color only; and a fallback instruction to rely on line-movement flags/fundamentals when splits are missing.

Interacts with: Intended to be concatenated/injected alongside `esm.market_intelligence.format_intelligence_for_prompt`'s output into the Claude user message context, reinforcing the same rules stated more tersely inline in `system_prompt.py`'s "SHARP-VS-PUBLIC INTERPRETATION" section.

`backend/esm/stats_client.py`

Purpose: Fetches supplementary stats context (scoreboards, player game logs, injury reports) from ESPN's unofficial/undocumented public API to ground Claude's fundamentals analysis in real data.

Key exports:

- `StatsClient` class with methods:
- `get_scoreboard(odds_sport_key, date_str=None)` — today's/target-date games with team records and embedded odds summary.
- `get_player_recent_stats(odds_sport_key, player_name, num_games=10)` — resolves a player name to an ESPN athlete ID then fetches their recent game log.
- `get_injuries(odds_sport_key)` — league-wide injury report (team, player, status, detail, return date).
- `build_context_package(odds_sport_key, date_str=None)` — bundles scoreboard + injuries into one dict.
- Internal: `_get`, `_find_athlete_id`, `_get_game_log`.
- `SPORT_MAP` — maps internal sport keys (`basketball_nba`, etc.) to ESPN's `(sport, league)` URL segments; notably **excludes soccer/World Cup** (only 5 of the 6 `ACTIVE_SPORTS` are covered).
- `ESPN_BASE`, `CDN_BASE`, `HEADERS`.

How it works: All methods hit `https://site.api.espn.com/apis/site/v2/sports/{sport}/{league}/...`

endpoints with a browser-spoofing `User-Agent`. `_find_athlete_id` does a linear substring name match over up to 1000 active athletes (case-insensitive, either-direction substring) — a fuzzy/imprecise lookup since ESPN's public athlete search isn't exact-match friendly. `_get_game_log` zips ESPN's parallel `categories[].labels` and `stats` arrays into a `{label: value}` dict per game, defensively falling back to an empty dict if lengths mismatch. All network errors are caught and logged, returning `None` / `[]` rather than raising, so a failed ESPN call degrades gracefully instead of blocking card generation.

Interacts with: Provides fundamentals/injury data merged into the Claude user-message context alongside `market_intelligence`'s output before the ESM system prompt's "ANALYSIS FRAMEWORK" steps (recent performance, injuries/availability) can be honored; independent of `odds_client.py` (different data domain: stats vs. prices).

Chapter 6: Application Services — backend/services/

This layer sits above `agent/` and `esm/` and implements the actual user-facing pipelines: generating and persisting the daily MLB/ESM card and the World Cup card, auto-grading bets against ESPN box scores, delivering cards by email, mirroring bet history to Google Sheets, and formatting cards as Twitter/X threads.

backend/services/__init__.py

Purpose: Marks `backend/services` as a Python package.

Key exports: None — the file is empty (a single blank line).

How it works: No logic; its mere presence lets other modules import from the `services` package (`services.card_store`, `services.mailer`, etc.) as seen throughout the other files.

Interacts with: Every other module in `backend/services/` implicitly, via Python's package import mechanism.

backend/services/agent_runner.py

Purpose: Generates the daily major-league (MLB/NBA/NHL/NFL) ESM betting card for a single user by assembling odds/injury/performance context, calling Claude, and persisting the resulting card.

Key exports:

- `run_card_for_user(user_id, prefs, target_date=None, *, force=False, market_snapshot=None) -> dict` — main entry point; builds context, calls Claude, enriches and persists the card, returns the `esm` card dict.
- `_build_market_snapshot(today, sport_keys=None) -> dict` — fetches odds via `OddsClient` for the requested sports.
- `_build_espn_context(market_snapshot, today) -> dict` — pulls ESPN injuries/scoreboard context per sport via `StatsClient`.
- `_build_user_message(...) -> str` — composes the full text prompt sent to Claude (market data, ESPN context, market intelligence, performance memory, instructions).
- `_summarize_market(snapshot) -> str` / `_summarize_espn(espn_context) -> str` — human-readable text renderers for the prompt.
- `_call_claude(user_message) -> dict` — invokes the Anthropic API with the ESM system prompt (cached) and parses the JSON response.
- `_repair_json(raw) -> dict` — best-effort salvage of a truncated/invalid JSON card by trimming to the last complete play and re-closing the JSON structure.

How it works: `run_card_for_user` first syncs "units at risk" and computes the user's unit size, then checks Supabase (`cards` table) for an existing card for that date/user to avoid duplicate generation unless `force=True`. It resolves the user's major-league sport keys, fetches (or reuses a supplied) market snapshot of odds, builds ESPN injury/scoreboard context, fetches prior performance memory (`learning.memory.get_performance_context`) and market intelligence (line movement/splits) which get attached to the snapshot. All of this is serialized into one large text prompt via `_build_user_message` and sent to Claude (model configured in `esm.claude_config`, with the ESM system prompt cached via `cache_control: ephemeral`). The raw response is stripped of markdown fences and parsed as JSON, with a fallback JSON-repair routine (`_repair_json`) if the model's output was truncated. The card is then enriched with market signal metadata, its `official_plays` truncated to `max_plays`, tagged with the date, and persisted through `services.card_store.persist_esm_card`.

Interacts with: `esm.claude_config`, `esm.odds_client.OddsClient`, `esm.stats_client.StatsClient`,

`esm.system_prompt.ESM_SYSTEM_PROMPT`, `learning.memory`, `esm.market_intelligence`, `esm.split_guidance`, `agent.unit_tracker`, `agent.sports`, `database.db` (Supabase `cards` / `preferences` tables), `services.card_store.persist_esm_card`, and the Anthropic API. Called by `services/daily_esm_card.py` and `main.py`'s scheduler / admin API endpoint.

`backend/services/card_store.py`

Purpose: Persists a generated ESM card (MLB/ESM or World Cup) and its official plays into Supabase's `cards` and `bets` tables, keeping the two pipelines' data separate within a shared card row.

Key exports:

- `resolve_user_id(db, email=None) -> Optional[str]` — looks up a `profiles.id` by case-insensitive email match (defaults to an env-configured address).
- `persist_esm_card(db, user_id, card, *, source="esm") -> Optional[str]` — upserts the card row (merging into `raw_card.esm` or `raw_card.world_cup`) and inserts any new, not-yet-seen bet rows; returns the card ID.
- `_normalize_game(game) -> str` — normalizes game strings (removes parenthetical suffixes, unifies "@"/"at"/"v" to "vs", lowercases) for de-dup comparisons.
- `_bet_key(play) -> tuple` — dedup key of (normalized game, lowercased bet text).
- `_slate_note_from_raw(raw) -> str` — builds a combined slate note string tagging `[World Cup]` and `[MLB/ESM]` notes.

How it works: `persist_esm_card` reads the card's `date`, checks existing bets for that user/day to build a `seen_bets` de-dup set, and checks for an existing `cards` row for that user/date. If a card row exists, it merges the new card under `raw_card[source]` (keeping World Cup and ESM/MLB payloads independent keys) and updates top-level dashboard columns (`plays`, `leans`, `quick_reads`, `pass_notes`, `slate_grade`) — with logic ensuring the MLB/ESM card takes precedence over World Cup for the shared top-level columns unless ESM data doesn't exist yet. If no card row exists, it inserts a new one. It then inserts one row per official play into `bets` (skipping already-seen games/bets), tagging each with `WC_BET_TAG` or `ESM_BET_TAG` from `agent.unit_tracker`, using default odds/book/units/confidence when absent. After inserting, it triggers `services.sheets_sync.maybe_sync_sheets` and (for non-World-Cup sources) `agent.unit_tracker.sync_units_at_risk`.

Interacts with: Supabase tables `profiles`, `cards`, `bets`; `agent.unit_tracker` (bet tag constants, unit sync); `services.sheets_sync`. Called by `agent_runner.py` (ESM cards) and `world_cup_card.py` (WC cards).

`backend/services/daily_esm_card.py`

Purpose: Orchestrates the end-to-end daily MLB/major-league card pipeline: refresh betting splits, generate the card via `agent_runner`, print/email it, and sync to Google Sheets. Runs separately from, and after, the World Cup pipeline.

Key exports:

- `today_mt() -> str` — today's date in the configured timezone (default America/Denver).
- `default_recipient() -> str` — recipient email from `ESM_CARD_USER_EMAIL` / `WC_CARD_USER_EMAIL` env vars.
- `esm_card_enabled() -> bool` — feature flag from `ESM_CARD_ENABLED` env var.
- `run_daily_esm_card(*, target_date=None, email=None, send_email=True, force=False, print_output=True, db=None) -> dict` — main pipeline entry point.
- `_esm_already_persisted(db, email, card_date) -> bool` — checks whether an ESM card with official plays already exists for the day.
- `_print_card(card) -> None` — console pretty-printer for the card (plays and leans).

How it works: `run_daily_esm_card` resolves the target date and recipient, exits early if the feature is disabled or (absent `force`) a card already exists. It runs Action Network splits sync (`workers.market_poller.run_splits_sync`), resolves

the recipient's Supabase user id and preferences, then delegates card generation to

`services.agent_runner.run_card_for_user`. If a card comes back, it optionally prints it to console, and if email is enabled and a transport is configured, sends it via `services.mailer.send_card_email` with MLB-specific subject/header labels. Finally it triggers `services.sheets_sync.maybe_sync_sheets`.

Interacts with: `services.mailer.send_card_email`, `services.world_cup_card.email_transport_configured`, `services.card_store.resolve_user_id`, `services.agent_runner.run_card_for_user`, `workers.market_poller.run_splits_sync`, `services.sheets_sync.maybe_sync_sheets`, Supabase `preferences` / `cards` tables. Invoked by a scheduler job (APScheduler, ~9:35 AM MT, after the ~8:50 AM World Cup card) or admin endpoint.

backend/services/grader.py

Purpose: Auto-grades pending bets by fetching final box scores/results from ESPN's public API, supporting player props (NBA/MLB/NHL/NFL) and soccer markets (moneyline, DNB, totals, BTTS), and refreshes downstream performance memory after grading.

Key exports:

- `grade_all_pending(db, *, as_of_date=None) -> dict` — top-level batch grader; grades all pending bets dated before the cutoff, updates `bets`, and triggers memory/units refresh.
- `recalculate_graded_units(db, user_id=None) -> dict` — recomputes `units_result` for all already-graded bets from `result` / `units` / `odds` to fix drift.
- `_grade_bet(bet) -> Optional[dict]` — dispatches a single bet to the soccer, game-total, or player-prop grading path based on sport/market/bet text.
- `_grade_soccer_bet`, `_grade_game_total_bet` — sport-specific grading logic returning `{result, units_result, tag}`.
- `_resolve_soccer_outcome(...)` — core soccer-market resolver (DNB, 3-way ML, totals, BTTS).
- Parsing helpers: `_parse_game_teams`, `_parse_dnb_team`, `_parse_ml_team`, `_parse_total`, `_parse_btts`, `_parse_bet`, `_extract_player`, `_team_matches` — regex-based extraction of bet semantics from free-text bet strings.
- `_find_event`, `_get_box`, `_get_player_stat`, `_get_match_scores`, `_batter_total_bases`, `_espn_get` — ESPN API lookups (scoreboard search, box score fetch, stat extraction, computed total bases when ESPN omits a TB column).
- Constants: `SPORT_MAP` (sport → ESPN sport/league path), `MARKET_TO_ESPN_STAT` (odds-market key → ESPN box-score stat label(s)), `SOCCER_MARKETS`.

How it works: `grade_all_pending` pulls all `bets` rows with `result="pending"` and `date < as_of_date`, and for each calls `_grade_bet`. That function classifies the bet (soccer vs. game total vs. player prop) using sport/market/bet-text heuristics, then locates the ESPN event via fuzzy team-name matching on the scoreboard endpoint for that date, fetches the box score/"summary" endpoint, and only proceeds if the game status is `post` (final). Player props are graded by regex-extracting the player name and Over/Under line from the bet string, then scanning the box score's athlete stat blocks for a matching name and summing the relevant stat columns (with special-case logic for pitcher outs from innings-pitched and batter total bases computed from hits/HR/2B/3B when ESPN doesn't expose a TB column directly). Soccer bets are resolved via `_resolve_soccer_outcome`, which tries DNB/Asian-handicap-0, then 3-way moneyline, then totals, then BTTS in sequence based on regex matches in the bet string. Every grading path returns W/L/P plus a `units_result` computed via `services.units.calculate_units_result` / `calculate_win_units`, and an optional diagnostic `tag` (e.g. "model miss", "close win", "exact total") that gets appended to `notes` or written to `post_slate_tag`. After updating each bet, `agent.post_grade.apply_post_grade_effects` runs, and once the batch completes, `learning.memory.refresh_memory` / `refresh_platform_memory` and `agent.post_grade.finalize_grade_batch` refresh performance memory for affected users, followed by unit-at-risk sync and a Google Sheets sync.

Interacts with: ESPN public API (`site.api.espn.com`), Supabase `bets` table, `services.units` (unit math), `agent.post_grade` (post-grade side effects), `agent.unit_tracker.sync_units_at_risk`, `learning.memory` (performance memory refresh), `services.sheets_sync.maybe_sync_sheets`. Invoked by the scheduled `morning_grade` job (~9:15 AM MT) and the `/api/admin/grade` / `/api/admin/grade-all` routes.

`backend/services/mailler.py`

Purpose: Renders a generated card (MLB or World Cup) into a styled HTML email and delivers it via SendGrid or SMTP, with a local-file fallback when no transport is configured.

Key exports:

- `build_html_email(card, card_date=None, *, header_label="DAILY CARD") -> str` — builds the full HTML document (dark-themed, inline-styled tables) for a card.
- `send_card_email(card, to_address, card_date=None, *, subject_label="ESM Daily Card", header_label="DAILY CARD") -> bool` — top-level send function; picks a transport and returns success/failure.
- `_play_html(play) -> str`, `_lean_html(lean) -> str`, `_section_bar(label, right_text, color) -> str`, `_sport_pill(sport) -> str` — HTML fragment builders for individual plays, leans, section headers, and sport badges.
- `_send_via_smtp(...) -> bool`, `_send_via_sendgrid(...) -> bool` — transport-specific senders.
- `_smtp_from_address() -> str` — resolves the SMTP "From" address, forcing it to the authenticated Gmail account when using Gmail SMTP (Gmail rejects mismatched From addresses) and setting a `Reply-To` for branding.
- Constants: `_CONFIDENCE_COLOR`, `_GRADE_COLOR`, `_GRADE_LABEL` — color/label lookup tables for confidence tiers and slate grades (A–F).

How it works: `build_html_email` assembles a single-column, email-client-safe HTML table layout: a gradient top accent, header with branding and a colored "slate grade" box, an "Official Plays" section (rendering each play's game/confidence/bet/odds/units/book/time plus implied vs. true probability vs. edge stat tiles, falling back to a "No official plays" placeholder), optional Leans/Quick Reads/Passes sections built conditionally only if data exists, and a footer disclaimer.

`send_card_email` builds the HTML and subject line, then checks for `SENDGRID_API_KEY` first, then `EMAIL_SMTP_HOST`; if neither is set, it writes the HTML to `/tmp/esm_card_<date>.html` and returns `False`. SMTP sending uses `smtplib` with STARTTLS and optional login; SendGrid sending uses a raw `urllib.request` POST to the SendGrid v3 mail API (no SDK dependency).

Interacts with: Environment variables for email configuration (`EMAIL_FROM`, `EMAIL_SMTP_*`, `SENDGRID_API_KEY`); called by `services/daily_esm_card.py` and `services/world_cup_card.py` to deliver generated cards.

`backend/services/sheets_sync.py`

Purpose: Exports all bets from Supabase into a Google Sheet (three tabs: All Bets, Overall Record, Record by Sport) whenever new bets are written or graded, giving a spreadsheet mirror of betting performance.

Key exports:

- `is_configured() -> bool` — true if `GOOGLE_SHEET_ID` and service-account credentials are present.
- `maybe_sync_sheets(db, *, reason="") -> Optional[dict]` — safe wrapper that no-ops (with a log) if unconfigured, and swallows/logs exceptions otherwise.
- `sync_bets_to_sheet(db) -> dict` — full sync: fetches bets, builds three tabs of rows, writes them via gspread, deletes obsolete legacy tabs (`Record`, `AgentEdge Bets`).
- `_fetch_bets`, `_fetch_profile_map`, `_bet_to_row`, `_build_overall_record_rows`, `_build_by_sport_rows`, `_calc_record` (back-compatible wrapper), `_write_worksheet`, `_delete_worksheet_if_exists`, `_get_gspread_client`, `_credentials_info` — row/formatting/auth helpers.

- Constants: `BETS_HEADERS` , `OVERALL_RECORD_HEADERS` , `BY_SPORT_HEADERS` .

How it works: `sync_bets_to_sheet` reads env-configured sheet ID and tab names, optionally filters bets to a single user by email (`GOOGLE_SHEETS_SYNC_EMAIL`), and converts each bet row into spreadsheet-ready values using `services.units` helpers for Units Won/Lost/Net columns. It computes lifetime and per-sport aggregate records via `services.units.aggregate_record` , authenticates to Google via a service-account JSON (`GOOGLE_SHEETS_CREDENTIALS_JSON` env var or a file path), opens the spreadsheet with `gsread` , and rewrites each worksheet in full (`clear()` then `update()`) rather than incrementally patching, with the header row frozen. Two legacy worksheet names are deleted if present, indicating a schema/naming migration.

Interacts with: Supabase `bets` and `profiles` tables; `services.units` (unit-math formatting/aggregation); Google Sheets API via `gsread` and `google.oauth2.service_account` . Called by `card_store.py` , `daily_esm_card.py` , `grader.py` , and `world_cup_card.py` after any bet-affecting event.

`backend/services/social.py`

Purpose: Converts a generated card dict into a ready-to-post Twitter/X thread (list of ≤ 280 -character tweet strings) for promotional distribution of the daily card; posting itself is left to the caller.

Key exports:

- `build_twitter_thread(card, card_date=None) -> list[str]` — builds the full thread: intro/grade tweet, one tweet per official play, an optional leans tweet, an optional quick-reads tweet, and a closing CTA tweet.
- `format_thread_for_display(tweets) -> str` — pretty-prints the thread (with char counts) for console/log output.
- `_truncate(text, max_len) -> str` , `_enforce_limit(tweet, limit=280) -> str` — text truncation helpers.
- Lookup constants: `_CONF_LABEL` , `_GRADE_LABEL` , `_SPORT_EMOJI` , `HASHTAGS_WORLD_CUP` , `HASHTAGS_DEFAULT` .

How it works: The function detects whether the card is World Cup content (any play/lean tagged sport `SOCCER`) to select World-Cup-specific hashtags/header/emoji vs. default sports-betting branding. It builds an intro tweet with the slate grade and note, then iterates official plays to build one emoji-decorated tweet per play (game, bet, odds, units, book, truncated edge summary, numbered "PLAY i/N"), followed by an optional leans tweet (capped at 4 items) and quick-reads tweet (capped at 4 items), and a closing CTA tweet whose wording also branches on World Cup vs. default. Every tweet is passed through `_enforce_limit` to hard-cap at 280 characters, truncating with an ellipsis if needed.

Interacts with: Consumes card dicts produced by `agent_runner.py` / `world_cup_card.py` ; called from `services/world_cup_card.py` (for console display via `format_thread_for_display`) — actual posting to the Twitter/X API is not implemented here.

`backend/services/splits_sync.py`

Purpose: Syncs public betting-splits data (bet% / money% / sharp indicators) from Action Network into the Supabase `market_splits` table, for use as market-intelligence input into card generation.

Key exports:

- `is_configured() -> bool` — reflects the `SPLITS_SYNC_ENABLED` env flag.
- `sync_splits_for_sport(db, sport_key) -> dict` — fetches and inserts split rows for one sport/league.
- `sync_all_active_splits(db, sport_keys) -> dict` — runs the sync across a given list of sport keys.
- `sync_all_mapped_splits(db) -> dict` — runs the sync across every sport Action Network supports.
- `find_splits_for_matchup(db, sport_key, away_team, home_team) -> dict` — reads back the latest splits per market for a specific matchup (fuzzy team-name matched, not by stored event ID).

How it works: `sync_splits_for_sport` checks the sport is enabled and mapped (`SPORT_TO_AN_PATH`), calls `esm.action_network_client.fetch_public_splits` to scrape/fetch Action Network's public betting page for that sport, and for each event/market with non-null ticket% or money% data, inserts a row into `market_splits` (including raw

metadata like odds/line/team/core_id/num_bets/captured_at as JSON). `find_splits_for_matchup` instead queries the most recent 500 `market_splits` rows for a sport and filters them by fuzzy team-name matching (`action_network_client.match_event_to_game`) against the given away/home teams, keeping only the newest row per market since results are pre-sorted by `captured_at desc`.

Interacts with: `esm.action_network_client` (Action Network scraping/matching logic), Supabase `market_splits` table. Called by `workers.market_poller.run_splits_sync`, and its `find_splits_for_matchup` is consumed by `esm.market_intelligence` when building card context.

`backend/services/units.py`

Purpose: Single source of truth for unit-based P/L math (stake, win profit, loss, push) shared across grading, card storage, and sheet sync so unit calculations stay consistent platform-wide.

Key exports:

- `calculate_win_units(units_risked, odds) -> float` — profit units on a win given American odds (positive or negative).
- `calculate_units_result(result, units_risked, odds) -> float` — net unit result for W/L/P.
- `normalize_units_result(bet) -> float` — recomputes a bet's `units_result` from its stored fields.
- `units_won(bet) -> float | None`, `units_lost(bet) -> float | None`, `net_units(bet) -> float | None` — per-bet derived metrics (None if not applicable/pending).
- `format_net_units`, `format_units_won`, `format_units_lost` — string formatters (`+1.23`, `1.23`, etc.) for display/sheets.
- `aggregate_record(bets) -> dict` — summarizes a list of graded bets into W-L-P record, units risked/won/lost, net units, and ROI%.

How it works: American-odds conversion follows standard sportsbook math: positive odds pay `units * (odds/100)`, negative odds pay `units * (100/abs(odds))`. Losses simply lose the full stake (`-units`); pushes are 0. `net_units` prefers a bet's already-stored `units_result` (for W/L/P) over recomputing, falling back to `normalize_units_result` otherwise — this lets manually-adjusted results survive. `aggregate_record` iterates bets, skips pending/ungraded ones, accumulates risked/won/lost/net totals and win/loss/push counts, then computes ROI as `net/risked100`.

Interacts with: * Used by `grader.py` (grading), `card_store.py` (implicitly via grader), and `sheets_sync.py` (row formatting and aggregate rows for Overall Record / Record by Sport tabs). No external I/O — pure functions operating on bet dicts.

`backend/services/world_cup_card.py`

Purpose: End-to-end pipeline for the daily FIFA World Cup 2026 ESM card: builds World Cup-specific odds/market-intelligence context, calls Claude with WC-specific prompt rules (DNB data-integrity, juice ceiling, correlation), validates/enriches the result, then prints, tweets-formats, emails, and persists it.

Key exports:

- `today_mt() -> str`, `default_recipient() -> str` — date/recipient helpers (env-driven, WC-specific email var).
- `email_transport_configured() -> bool` — whether SMTP/SendGrid is set up (also imported by `daily_esm_card.py`).
- `run_world_cup_card(*, target_date=None, email=None, send_email=True, persist=True, max_plays=5, unit_size=None, print_output=True, force=False, db=None) -> dict` — main pipeline entry point.
- `_wc_tournament_day(target_date) -> int` — computes tournament day number from `WC_START_DATE`.
- `_build_wc_market_snapshot(target_date, db=None) -> dict` — prefers a pre-cached morning odds snapshot, falls back to a live TheOddsAPI (TOA) fetch.

- `_build_wc_user_message(...)` -> `str` — builds the Claude prompt including WC-specific DNB integrity rules and instructions.
- `_call_claude(user_message)` -> `dict` — calls Anthropic API (no salvage/repair logic here — raises on parse failure, unlike `agent_runner.py`).
- `_print_card(card)` -> `None`, `_summarize_market(snapshot)` -> `str` — console/prompt formatting.
- `_resolve_db(db=None)`, `_card_already_exists(db, email, card_date)` -> `bool`, `_persist_card(card, email, db=None)` -> `None` — DB resolution, dedup check, and persistence via `card_store`.
- Constants: `WC_SPORT_KEY = "soccer_fifa_world_cup"`, `WC_START_DATE = date(2026, 6, 11)`.

How it works: `run_world_cup_card` resolves the target date/recipient/DB client, and unless `force=True`, skips generation if a World Cup card already exists for that user/date (returning the cached `raw_card.world_cup` payload). It determines unit size from the user's bankroll context (falling back to a flat \$30/unit default), fetches the WC market snapshot (preferring a cached snapshot written earlier in the morning, else a live TheOddsAPI fetch), runs Action Network splits sync, and attaches market intelligence (line movement/steam/reverse-line/splits) to the snapshot. It composes a detailed WC-specific prompt (`_build_wc_user_message`) that enforces strict Draw-No-Bet data-integrity rules (never estimate DNB from 3-way ML, respect a -150 juice ceiling, never fabricate missing lines) and instructs Claude to weigh fundamentals first, then market intelligence, then price. The Claude response is parsed as JSON (raising on failure — this pipeline has no truncation-repair fallback), tagged with the date, then validated against actual posted odds via

`esm.soccer_odds.validate_wc_official_plays` and enriched with market signals. If enabled, it prints the card and a formatted Twitter thread (via `services.social`), emails it via `services.mailer.send_card_email` with WC-specific subject/header labels, and persists it via `_persist_card` → `services.card_store.persist_esm_card(..., source="world_cup")`, followed by a Google Sheets sync.

Interacts with: `esm.claude_config`, `esm.odds_client.OddsClient`, `esm.system_prompt.ESM_SYSTEM_PROMPT`, `esm.snapshot_cache.get_cached_snapshot`, `esm.market_intelligence`, `esm.split_guidance`, `esm.soccer_odds.validate_wc_official_plays`, `services.mailer.send_card_email`, `services.social` (Twitter thread), `services.card_store` (persistence, user resolution), `agent.unit_tracker.get_unit_context`, `workers.market_poller.run_splits_sync`, `services.sheets_sync.maybe_sync_sheets`, Anthropic API, Supabase `cards` table. Runs on its own schedule (GitHub Actions cron primary + Railway APScheduler backup at ~8:50 AM MT), ahead of the MLB/ESM pipeline in `daily_esm_card.py`.

Chapter 7: Core Backend, Learning Loop, Workers & Scripts

This chapter covers the FastAPI application itself (`main.py`) — every API route and every scheduled job — plus authentication, the Supabase client singleton, the dual-brain learning/memory module, the shared market-polling worker, the standalone CLI entry points, and deployment/maintenance scripts.

`backend/main.py`

Purpose: FastAPI application entrypoint for the AgentEdge API — exposes REST endpoints for per-user agents, daily betting cards, bets, preferences, and admin operations, and runs all scheduled background jobs (card generation, market polling, grading, memory refresh) via APScheduler.

Key exports:

- `app` (FastAPI instance, title "AgentEdge API", v2.0.0) — the ASGI app served by uvicorn.
- `scheduler` (AsyncIOScheduler) — module-level scheduler instance, timezone-aware (`TIMEZONE`, default `America/Denver`).
- `lifespan(app)` — async context manager that registers all cron/interval jobs on startup and shuts the scheduler down on exit.
- `RegisterRequest`, `PreferencesUpdate`, `AgentSetupRequest`, `GradeRequest` — Pydantic request models.
- `run_daily_cards()`, `run_weekly_digest()`, `_calculate_record()` — helper coroutines/functions used by scheduled jobs and admin routes.
- `_expand_card_row()` — reshapes a raw `cards` row into separate `world_cup_card` / `esm_card` / `mlb_plays` / `world_cup_plays` fields for the frontend.

How it works: On startup, `lifespan()` logs scheduler config and registers 11 jobs on the AsyncIOScheduler:

- **Cron jobs:** `morning_grade` (9:15 — grade yesterday's pending bets + sync unit exposure), `morning_toa_snapshot` (8:40 — force a full Odds API snapshot before the WC card), `morning_splits_sync` (8:45), `world_cup_card` (8:50 — generate/email/persist the WC card, ensuring fresh odds first), `pre_agent_splits_sync` (9:25), `morning_agent_run` (9:30 — re-sync unit exposure ahead of scans), `daily_esm_card` (9:35 — generate the MLB/major-league ESM card), `sheets_sync` (10:00 — push graded bets to Google Sheets), `weekly_digest` (Mondays 8:00 — per-user weekly performance recap).
- **Interval jobs:** `market_poll` every `POLL_INTERVAL_MINUTES` (odds polling into snapshot cache) and `agent_scans` every `AGENT_SCAN_INTERVAL_MINUTES` (runs Claude-driven scans for all active agents, gated by `AGENT_SCANS_ENABLED`).

Routes are grouped into: auth (`/auth/register` — validates an invite code, creates a Supabase auth user, seeds preferences), user data (`/api/card/today`, `/api/card/{date}`, `/api/bets`, `/api/record`, `/api/record/daily`, `/api/preferences` GET/PUT, `/api/profile`, `/api/sports`), agent management (`/api/agent`, `/api/agent/setup`, `/api/agent/feed`, `/api/agent/scan`, `/api/agent/pause`, `/api/agent/resume`), and admin-only endpoints protected by `get_admin_user` (`/api/admin/run-card`, `/api/admin/run-esm-card`, `/api/admin/run-wc-card`, `/api/admin/grade`, `/api/admin/weekly-digest`, `/api/admin/grade-all`, `/api/admin/backfill-bankroll`, `/api/admin/refresh-memory`, `/api/admin/recalculate-units`, `/api/admin/sync-sheets`, `/api/admin/users`, `/api/admin/invite`, `/api/admin/pending-bets`, `/api/admin/morning-toa-poll`, `/api/admin/api-usage`). A `/health` route supports Railway's healthcheck. CORS origins are derived from a comma-separated `FRONTEND_URL` env var (always

including localhost:3000). Most heavy synchronous work (card generation, sheet sync) is offloaded via `asyncio.to_thread`.

Interacts with: `database.db` (Supabase client), `auth.get_current_user` / `get_admin_user`, `services.grader`, `services.agent_runner`, `services.world_cup_card`, `services.daily_esm_card`, `services.sheets_sync`, `services.units`, `agent.provision`, `agent.kernel`, `agent.sports`, `agent.unit_tracker`, `agent.post_grade`, `agent.bankroll_backfill`, `agent.memory_store`, `workers.market_poller`, `esm.api_budget`, `esm.claude_config`, `esm.odds_client`, `learning.memory`, and Supabase tables `profiles`, `preferences`, `cards`, `bets`, `invite_codes`, `agent_instances`.

backend/auth.py

Purpose: Supabase-based authentication/authorization dependency module used by every protected FastAPI route.

Key exports:

- `security` — module-level `HTTPBearer` instance for extracting bearer tokens.
- `get_current_user(credentials)` — async FastAPI dependency that validates a Supabase JWT and returns `{"id", "email"}`.
- `get_admin_user(user)` — async dependency chained on `get_current_user`; additionally checks `profiles.is_admin`.

How it works: `get_current_user` builds a fresh Supabase client using the anon key and calls `client.auth.get_user(token)`; if the token is invalid/expired it raises `HTTPException(401)`. `get_admin_user` depends on `get_current_user`, then queries the `profiles` table for the caller's `is_admin` flag (using the shared service-role `db` client from `database.py`), raising `HTTPException(403)` if not set.

Interacts with: `database.db` (for the admin check), Supabase Auth API, and is consumed by nearly every route in `main.py`.

backend/database.py

Purpose: Centralized Supabase client factory/singleton shared across the entire backend.

Key exports:

- `get_service_client()` — returns a Supabase `Client` authenticated with the service-role key (bypasses RLS).
- `get_anon_client()` — returns a Supabase `Client` authenticated with the anon key (respects RLS).
- `db` — module-level singleton, the result of `get_service_client()`; this is the object imported everywhere else as "the database."

How it works: Loads `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`, `SUPABASE_ANON_KEY` from the environment via `python-dotenv`, then eagerly constructs a service-role client at import time and exposes it as `db`.

Interacts with: Supabase (Postgres + Auth backend). Imported by `main.py`, `auth.py`, all scripts, `workers/market_poller.py`, `learning/memory.py`, and most `services` / `agent` modules.

backend/run_daily_esm_card.py

Purpose: Standalone CLI entrypoint to manually generate (and optionally email) the MLB/major-league "ESM" daily card outside of the scheduler.

Key exports: `main()` — parses CLI args (`--email`, `--date`, `--no-email`, `--force`) and invokes `run_daily_esm_card`.

How it works: Inserts its own directory onto `sys.path`, then delegates entirely to `services.daily_esm_card.run_daily_esm_card`, defaulting the recipient to `default_recipient()` and the date to

`today_mt()` (Mountain Time "today"). Simple argparse wrapper with no business logic of its own.

Interacts with: `services.daily_esm_card` (`default_recipient`, `run_daily_esm_card`, `today_mt`).

`backend/run_world_cup_card.py`

Purpose: Standalone CLI entrypoint to manually generate the World Cup daily card, independent of both the Railway APScheduler job and the GitHub Actions cron that primarily drives it.

Key exports: `main()` — parses CLI args (`--email`, `--date`, `--max-plays`, `--unit-size`, `--no-email`, `--no-persist`, `--force`) and calls `run_world_cup_card`.

How it works: Adds its directory to `sys.path` and thinly wraps `services.world_cup_card.run_world_cup_card`, passing through card-sizing (`max_plays`, `unit_size`) and delivery/persistence toggles. This is the third delivery path for the WC card — GitHub Actions cron is primary, Railway APScheduler is backup, this script is manual.

Interacts with: `services.world_cup_card` (`default_recipient`, `run_world_cup_card`, `today_mt`).

`backend/sync_sheets.py`

Purpose: One-shot manual CLI to push all Supabase `bets` rows into the configured Google Sheet bet tracker.

Key exports: `main()` — checks configuration, runs the sync, and prints a summary.

How it works: Loads env vars, imports the shared `db` client, and checks `services.sheets_sync.is_configured()` (requires `GOOGLE_SHEET_ID` + credentials); if not configured it prints an error and exits with code 1. Otherwise it calls `sync_bets_to_sheet(db)` and prints the row count and sync timestamp returned.

Interacts with: `database.db`, `services.sheets_sync` (`is_configured`, `sync_bets_to_sheet`), Google Sheets API (via `gspsread` / `google-auth`, indirectly).

`backend/learning/_init__.py`

Purpose: Package marker for the `learning` module. Empty (0 bytes) — no logic, just makes `learning/` importable.

`backend/learning/memory.py`

Purpose: Implements AgentEdge's "dual-brain" performance-memory system: per-user historical stats (primary brain) and platform-wide collective stats (secondary brain), both persisted to Supabase and formatted for injection into Claude prompts so the agent can calibrate future recommendations on past results.

Key exports:

- `refresh_memory(db, user_id)` — recomputes and upserts one user's stats into `agent_memory`.
- `refresh_platform_memory(db)` — recomputes and upserts platform-wide stats into `platform_memory` (key `"global"`).
- `refresh_memory_all_users(db)` — refreshes every user with graded bets, then the platform brain; returns a summary dict.
- `get_performance_context(db, user_id, pipeline="agent")` — builds the full text block (user memory + platform memory) for prompt injection.
- `LOOKBACK_DAYS` (90), `PIPELINE_TAGS` (`"agent"`, `"esm"`, `"world_cup"`), `PLATFORM_MEMORY_KEY` (`"global"`) — module constants.
- Internal helpers: `_compute_user_stats`, `_compute_platform_stats`, `_aggregate_bets`, `_weak_markets_platform`, `_tally`, `_odds_bucket`, `_format_user_memory`, `_format_platform_memory`, `_format_summary_block`, `_format_breakdowns`, `_format_platform_breakdowns`.

How it works: Stats are computed over a rolling 90-day window from the `bets` table (excluding `pending`).

`_aggregate_bets` tallies wins/losses/pushes, net units (via `services.units.normalize_units_result`), and breakdowns by market, sport, confidence tier, and an odds bucket (`heavy_fav/big_fav/fav/pick/dog/big_dog`), plus a rolling list of the 10 most recent losses (user-level only — platform stats never expose individual bet text, only anonymized market-level "weak markets" via `_weak_markets_platform`). Bets are also split by `_pipeline_key` (`agent / esm / world_cup / other`, from `post_slate_tag`) so per-pipeline stats can be surfaced. `get_performance_context` fetches (or lazily computes) both brains and stitches together a formatted text block, explicitly instructing the model that "when user memory and platform brain conflict, ALWAYS trust the user's own history."

Interacts with: Supabase tables `bets`, `agent_memory`, `platform_memory`; `services.units.normalize_units_result`; called from `main.py` admin routes (`/api/admin/grade`, `/api/admin/refresh-memory`) and `scripts/production_setup.py`; consumed by the agent's Claude prompt-building code via `get_performance_context`.

`backend/workers/_init_.py`

Purpose: Package marker/docstring (`# AgentEdge background workers`) for the `workers` module. No logic.

`backend/workers/market_poller.py`

Purpose: Shared "market perception" layer — polls betting odds once centrally and caches snapshots so that all per-user agent scans read from cache instead of hitting the odds APIs directly (protecting API quota).

Key exports:

- `poll_markets(db, force_source=None)` — fetches and stores market snapshots for all in-season/active sports; respects quota reserves.
- `run_splits_sync(db)` — pulls Action Network public/money betting-percentage splits for all mapped sports.
- `poll_morning_toa_snapshot(db)` — forces a full-props snapshot from The Odds API (TOA) at ~8:40 AM MT, ahead of the WC card and agent morning scan; also triggers a splits sync.
- `ensure_wc_odds_before_card(db)` — guarantees fresh World Cup odds (polls TOA if the cache is stale/missing, threshold `WC_ODDS_MAX_AGE_MINUTES`, default 45 min).
- `sync_wc_splits_before_card(db)` — thin alias for `run_splits_sync`.
- `run_all_agent_scans(db)` — iterates every active+provisioned `agent_instances` row and runs `agent.kernel.run_agent_scan` for each.
- Constants: `TIMEZONE`, `SPLITS_SYNC_ON_POLL`, `WC_SPORT_KEY` (`soccer_fifa_world_cup`), `WC_ODDS_MAX_AGE_MINUTES`.

How it works: `poll_markets` determines active sports via `agent.sports.get_active_sports_for_polling`, checks that at least one odds API key is configured, and — unless quota is critically low (checked via `esm.api_budget.should_use_sgo / should_use_toa`) — builds a market snapshot through `OddsClient.build_market_snapshot` and persists per-sport data via `esm.snapshot_cache.store_snapshot`. `run_all_agent_scans` is the fan-out point that triggers Claude-driven per-user scans reading from this cache rather than calling odds APIs themselves — a deliberate design to centralize and rate-limit external API usage.

Interacts with: `agent.kernel.run_agent_scan`, `agent.sports`, `esm.api_budget`, `esm.odds_client.OddsClient`, `esm.snapshot_cache`, `services.splits_sync`; Supabase table `agent_instances`; invoked from `main.py`'s scheduled jobs and `scripts/correct_wc_card_dnb.py`.

backend/scripts/backfill_bankroll.py

Purpose: One-time/on-demand CLI to rebuild `agent_instances.bankroll_current` by replaying an agent's entire graded-bet history — used to repair drift or bootstrap bankroll after a fix.

Key exports: `main()` — parses `--user-id` (optional; defaults to all agent instances) and prints a per-user before/after bankroll summary.

How it works: Loads `env`, imports `database.db` and `agent.bankroll_backfill` lazily. If `--user-id` is given it calls `replay_bankroll_for_user`; otherwise `backfill_all_agent_bankrolls` processes every agent, printing skips/errors/successes with starting vs. current bankroll, bet count replayed, and net P&L.

Interacts with: `database.db`, `agent.bankroll_backfill`; also invoked via `main.py`'s `/api/admin/backfill-bankroll` and part of `scripts/production_setup.py`'s pipeline.

backend/scripts/correct_wc_card_dnb.py

Purpose: One-off remediation script to fix an already-published World Cup card that contained invalid "draw no bet" (DNB) plays — re-validates against live posted odds, purges bad bets from the DB, updates the stored card, and resends the corrected email.

Key exports: `main()` — parses `--date` (defaults to today MT) and `--no-email`; performs the full correction workflow and returns an exit code.

How it works: Loads the target date's `cards` row for the WC recipient user, extracts the `raw_card.world_cup` section, fetches a fresh TOA-forced odds snapshot for `soccer_fifa_world_cup`, stores it in the snapshot cache, then calls `esm.soccer_odds.validate_wc_official_plays` to re-derive corrected official plays. It deletes any `bets` rows with `market = "draw_no_bet"` for that date/user, updates matching totals bets' odds/book to the corrected values, and updates the `cards` row's `plays`, `slate_note`, `pass_notes`, and `raw_card`. Finally it triggers a Google Sheets sync and (unless `--no-email`) resends the corrected card.

Interacts with: `database.db`, `esm.odds_client.OddsClient`, `esm.soccer_odds.validate_wc_official_plays`, `esm.snapshot_cache.store_snapshot`, `services.card_store.resolve_user_id`, `services.mailer.send_card_email`, `services.sheets_sync.maybe_sync_sheets`, `services.world_cup_card`; Supabase tables `cards`, `bets`, `profiles`.

backend/scripts/production_setup.py

Purpose: Post-deploy/maintenance CLI that runs a batch of housekeeping tasks in sequence (grade pending bets, backfill bankrolls, refresh memory, sync sheets, seed platform memory row) — designed to be triggered by a GitHub Actions "Production Setup" workflow using only service-role credentials (no admin JWT).

Key exports: `main()` — orchestrates the maintenance sequence and returns an exit code.

How it works: Verifies `SUPABASE_URL` / `SUPABASE_SERVICE_KEY` are present, then sequentially: (1) `grade_all_pending(db)`, (2) `backfill_all_agent_bankrolls(db)`, (3) `refresh_memory_all_users(db)`, (4) syncs the Google Sheet if configured (swallowing errors), and finally upserts a `platform_memory` row with key `"global"` if missing. Each step prints its result; failures in the sheet-sync step are caught and logged without aborting the run.

Interacts with: `database.db`, `services.grader.grade_all_pending`, `agent.bankroll_backfill.backfill_all_agent_bankrolls`, `learning.memory.refresh_memory_all_users`, `services.sheets_sync`; Supabase table `platform_memory`; referenced by `.github/workflows/production-setup.yml`.

backend/Dockerfile

Single build on `python:3.12-slim`. Installs `gcc` / `libpq-dev` (needed for `cryptography` / Postgres wheels), installs `requirements.txt`, copies the app, and runs `uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}`. Sets `PYTHONDONTWRITEBYTECODE` / `PYTHONUNBUFFERED`.

backend/railway.toml

Railway deployment config: builds via this Dockerfile, healthchecks `/health` with a 30s timeout, restarts on failure (`restartPolicyType = "ON_FAILURE"`).

backend/requirements.txt

Pinned Python dependencies: `fastapi`, `uvicorn[standard]`, `supabase`, `python-dotenv`, `anthropic` (Claude SDK, `>=0.40.0`), `requests`, `python-jose[cryptography]`, `python-multipart`, `apscheduler`, `httpx`, `pydantic`, `gspread`, `google-auth`.

backend/.env.example / backend/.env.template

Two annotated example env files covering Supabase, Anthropic, dual odds-API providers (SGO primary / TOA fallback) with quota/reserve thresholds, polling/agent-scan intervals, Claude cost-control knobs, unit-sizing defaults (3%/6%), WC card + email SMTP settings, and Google Sheets sync config. `.env.template` is the leaner minimal-bootstrap version; `.env.example` is the fuller annotated version.

backend/.python-version / backend/runtime.txt

Pin the Python version to 3.12 (`.python-version: 3.12`; `runtime.txt: python-3.12.8`), consistent with the Dockerfile's `python:3.12-slim` base.

Chapter 8: Frontend Application — frontend/

A Next.js 14 (App Router) single-page application, styled with Tailwind CSS in a dark, monospace, terminal-style aesthetic (JetBrains Mono, a custom edge color palette). It talks to the FastAPI backend over a typed fetch wrapper and to Supabase directly for authentication.

app/agent/page.tsx

Purpose: Client page at route /agent — the "agent cockpit," showing the live status, bankroll, and reasoning feed of the user's AI betting agent.

Key elements: useState / useEffect / useCallback hooks; local Episode, Hypothesis, Belief, AgentStatus interfaces; tabs for feed / watching / beliefs; a timeAgo() helper; "RUN SCAN NOW" button; calls api.getAgent, api.getAgentFeed, api.triggerScan.

How it works: On mount it grabs the Supabase session token, redirects to /login if absent, then loads agent status + feed in parallel; if the agent isn't provisioned it redirects to /setup. It renders a bankroll stat bar (current bankroll, P&L, unit size, daily exposure), a mode indicator, and three tabs: a live reasoning feed (episodes color-coded by type), watched hypotheses the agent is tracking before acting, and learned "beliefs"/memory with confidence scores. The "RUN SCAN NOW" button manually triggers a scan and reloads data.

Interacts with: Supabase auth.getSession(); backend GET /api/agent, GET /api/agent/feed, POST /api/agent/scan.

app/auth/callback/route.ts

Purpose: Next.js route handler (GET /auth/callback) that completes Supabase's PKCE auth flow (password reset / OAuth redirects) server-side.

Key elements: createServerSupabaseClient(); reads code and next query params; validates next is a safe relative path.

How it works: Extracts the code param from the callback URL; if missing, redirects to /reset-password?error=missing_code. Otherwise calls supabase.auth.exchangeCodeForSession(code) to set the session cookie; on error redirects to /reset-password with the error message, on success redirects to safeNext (default /dashboard).

Interacts with: Supabase server client auth.exchangeCodeForSession.

app/dashboard/page.tsx

Purpose: Client page at /dashboard — the "POSITIONS" view showing today's generated betting card (recommended plays) and overall win/loss record.

Key elements: Play / Card interfaces; StatBox and expandable PlayCard subcomponents; LoadingScreen; state for card, record, loading, error, token; calls api.getTodayCard, api.getRecord.

How it works: On mount, verifies the Supabase session (redirects to /login if none), then fetches today's card and the all-time record in parallel. Displays a record strip (W-L-P, net units, ROI, total graded), then renders sections for World Cup plays, MLB/ESM plays (each with slate grade and expandable per-play analysis: edge summary, line movement, sharp action, injury news), plus "Leans," "Quick Reads," and "Pass Notes" sections. Handles legacy card format (single combined slate) as a fallback. Includes a sign-out button.

Interacts with: Supabase `auth.getSession()`, `auth.signOut()`; backend `GET /api/card/today`, `GET /api/record`.

`app/globals.css`

Purpose: Global stylesheet importing Tailwind layers and the JetBrains Mono font, establishing the app's dark terminal-style visual theme.

Key elements: Google Fonts import, `@tailwind` directives, body background/color/font, custom scrollbar styling, `blink` and `fadeIn` keyframe animations, `.glow-green` text-shadow utility.

`app/history/page.tsx`

Purpose: Client page at `/history` showing the full bet history log grouped by date, alongside all-time performance record.

Key elements: `Bet` interface; `resultColor` / `resultBg` helpers; state for `bets`, `record`, `loading`, `error`; calls `api.getBets`, `api.getRecord`.

How it works: On mount checks Supabase session (redirect to `/login` if none), then fetches up to 200 bets plus the record summary. Groups bets by date (newest first), computing per-day win/loss counts and net units. Renders an all-time record summary card followed by date-grouped bet cards showing bet description, game, result (W/L/P/pending), odds, units risked, and units won/lost, color-coded by outcome.

Interacts with: Supabase `auth.getSession()`; backend `GET /api/bets?limit=200`, `GET /api/record`.

`app/layout.tsx`

Purpose: Root layout for the Next.js App Router; wraps every page with global HTML/body structure, metadata, and the global auth-hash handler.

Key elements: `metadata` export; renders `<AuthHashHandler />` before `{children}`.

How it works: Sets page-wide styling (dark background, mono font) and mounts `AuthHashHandler` globally so any page can receive/process Supabase auth redirect hashes.

`app/login/page.tsx`

Purpose: Client page at `/login` for signing in existing users, with an inline "forgot password" flow.

Key elements: Email/password form state; `showForgot` toggle; `handleLogin`, `handleForgotPassword`; uses `createClient`, `getSupabaseConfig` / `CONFIG_ERROR` / `getAppUrl` from `lib/env`, and `redirectAfterAuth` from `lib/auth-routing`.

How it works: If a session already exists on mount, immediately redirects via `redirectAfterAuth`. On submit, calls Supabase `signInWithPassword`; on success routes the user based on whether their agent is provisioned. The forgot-password form calls `supabase.auth.resetPasswordForEmail` with a redirect to `/auth/callback?next=/reset-password`. Displays config errors if Supabase env vars are missing, and links to `/register`.

Interacts with: Supabase `auth.getSession()`, `auth.signInWithPassword()`, `auth.resetPasswordForEmail()`; indirectly backend `GET /api/agent` (via `redirectAfterAuth`).

`app/page.tsx`

Purpose: Public marketing/landing page at `/` introducing AgentEdge (by EdgeSportsMedia) as an invite-only AI sports betting agent product.

Key elements: Static presentational component only — header with sign-in link, hero heading, feature grid (Live Feed, Auto

Bankroll, Living Memory, Your Sports), CTA buttons to `/register` and `/login`, footer disclaimer.

How it works: Purely static marketing content, no data fetching or auth logic.

`app/preferences/page.tsx`

Purpose: Client page at `/preferences` for editing an already-provisioned agent's ongoing betting preferences (sports, bet types, risk, sizing, notifications).

Key elements: `prefs` state object (sports, bet_types, risk_level, max_plays, unit_size, include_parlays, notification_email); toggle buttons/sliders; `handleSave`; calls `api.getPreferences`, `api.updatePreferences`.

How it works: On mount, verifies session and loads existing preferences, falling back to defaults. Users toggle sports (MLB/NBA/NHL/NFL/WC), bet types, risk level, adjust max plays per day and unit size via slider/input, toggle "include parlays," and set an optional notification email. "SAVE PREFERENCES" persists changes via PUT, showing a temporary "SAVED ✓" confirmation.

Interacts with: Supabase `auth.getSession()`; backend `GET /api/preferences`, `PUT /api/preferences`.

`app/register/page.tsx`

Purpose: Client page at `/register` for creating a new account using an invite code (invite-only beta).

Key elements: `form` state (full_name, email, password, invite_code); `handleRegister`; uses `apiPostPublic` from `lib/api`.

How it works: Renders a form generated from a field-config array; on submit, normalizes inputs (lowercased email, uppercased invite code) and POSTs to the public `/auth/register` backend endpoint. On success redirects to `/login?registered=1`; displays errors otherwise.

Interacts with: Backend `POST /auth/register` (unauthenticated).

`app/reset-password/page.tsx`

Purpose: Client page at `/reset-password` where users set a new password after clicking a Supabase password-recovery email link; wraps content in `<Suspense>` due to `useSearchParams`.

Key elements: `ResetPasswordForm` inner component; `VerifyState` (`loading` / `ready` / `error`); handles four possible recovery-link formats (hash tokens, query OTP `token_hash`, PKCE `code`, or pre-established session); a 12s timeout fallback; `handleSubmit` calls `supabase.auth.updateUser`.

How it works: On mount, attempts multiple strategies in sequence to establish a valid recovery session (hash tokens, OTP, PKCE code, or existing session), listening for Supabase `PASSWORD_RECOVERY` / `SIGNED_IN` auth events too. Once verified, shows a form to enter and confirm a new password (min 8 chars); on submit, updates the password and routes onward via `redirectAfterAuth`.

Interacts with: Supabase `auth.setSession()`, `auth.verifyOtp()`, `auth.exchangeCodeForSession()`, `auth.getSession()`, `auth.onAuthStateChange()`, `auth.updateUser()`.

`app/setup/page.tsx`

Purpose: Client page at `/setup` — a 3-step onboarding wizard that provisions a new user's AI betting agent (bankroll, sports/markets, risk profile).

Key elements: `step` state (1–3); `form` state (bankroll_starting, unit_pct, max_daily_pct, sports, bet_types, risk_level, max_plays, include_parlays, notification_email); `sports` list fetched from backend; derived `unitSize` / `maxUnits` calculations; calls `api.getAgent`, `api.getSports`, `api.setupAgent`.

How it works: On mount, checks session and whether the agent is already provisioned (redirects to `/agent` if so); fetches

the list of available sports. Step 1 sets starting bankroll and unit-size/max-daily-exposure percentages (auto-calculating dollar unit size). Step 2 picks sports and bet types. Step 3 sets risk profile, max plays/day, notification email, shows a summary, and "LAUNCH AGENT" calls `api.setupAgent` then routes to `/agent`.

Interacts with: Supabase `auth.getSession()`; backend `GET /api/agent`, `GET /api/sports`, `POST /api/agent/setup`.

`components/AuthHashHandler.tsx`

Purpose: Global client component (mounted in root layout) that intercepts Supabase auth redirect URL hashes (`#access_token=...`) app-wide and establishes sessions or reroutes accordingly.

How it works: On every route change, parses the URL hash for auth tokens. If the hash type is `recovery`, redirects to `/reset-password` preserving the hash. Otherwise calls `supabase.auth.setSession()` with the tokens, clears the hash from the URL, and calls `redirectAfterAuth` to send the user to `/agent` or `/setup`.

Interacts with: Supabase `auth.setSession()`; indirectly backend `GET /api/agent` via `redirectAfterAuth`.

`lib/api.ts`

Purpose: Central typed API client wrapping all `fetch` calls to the FastAPI backend, handling auth headers and error parsing.

Key elements: `parseResponse()` (JSON parsing with HTML-error detection), `apiPostPublic()` (unauthenticated POST), `apiFetch()` (authenticated fetch with Bearer token), and the exported `api` object containing all endpoint methods.

How it works: `apiFetch` attaches `Authorization: Bearer <token>` and JSON content-type headers to every authenticated call and throws a normalized `Error` (using `detail` / `message` fields, or a generic "backend unreachable" message if HTML is returned instead of JSON). The `api` object exposes methods for cards, bets, record, profile, preferences, sports, agent status/setup/feed/scan/pause/resume, and admin operations.

Interacts with: All backend `/api/*` and `/auth/register` endpoints listed in Chapter 7.

`lib/auth-routing.ts`

Purpose: Shared helper module for post-authentication routing decisions and parsing Supabase auth redirect URL hash fragments.

Key elements: `redirectAfterAuth(token, router, type)`, `parseAuthHash()`, `clearAuthHash()`.

How it works: `redirectAfterAuth` sends password-recovery flows to `/reset-password`; otherwise calls the backend to check `api.getAgent(token).provisioned` and routes to `/agent` if true or `/setup` if false/on error.

Interacts with: Backend `GET /api/agent`.

`lib/env.ts`

Purpose: Environment-variable accessor module centralizing Supabase config, backend API URL, and app URL resolution, plus a shared config-error message.

Key elements: `getSupabaseConfig()`, `getApiUrl()`, `getAppUrl()`, exported `CONFIG_ERROR` string constant.

How it works: Reads

`NEXT_PUBLIC_SUPABASE_URL` / `NEXT_PUBLIC_SUPABASE_ANON_KEY` / `NEXT_PUBLIC_API_URL` / `NEXT_PUBLIC_APP_URL`, with sensible localhost fallbacks.

`lib/supabase.ts` / `lib/supabase/server.ts`

Purpose: Factories for creating Supabase clients — `supabase.ts` for browser/client components

(`createBrowserClient`), `supabase/server.ts` for Route Handlers/Server Components (`createServerClient` wired

to Next.js `cookies()`).

How it works: Both throw if Supabase config is missing. The server variant wraps `setAll` in try/catch since cookie mutation can fail in Server Components (used by the `/auth/callback` route to exchange a PKCE code for a session cookie).

`next.config.js` / `package.json` / `postcss.config.js` / `tailwind.config.ts` / `tsconfig.json`

Minimal Next.js config (no custom settings). `package.json` defines the `edgebet-frontend` Next.js 14.2.5 / React 18 app (key deps: `@supabase/supabase-js`, `@supabase/ssr`, `lucide-react`, `clsx`). `tailwind.config.ts` extends the theme with a custom `edge` dark-terminal color palette and `JetBrains Mono` as the default mono font. `tsconfig.json` is standard strict Next.js TypeScript config with the `@/*` path alias.

`public/card-preview.html`

A static standalone HTML email-preview template (not part of the Next.js routes) showing what the "ESM Daily Card" email looks like — dark-themed table-based HTML email markup with a sample slate grade and sample plays, used for design preview at `/card-preview.html` on Vercel.

`.env.example`

Template for local `.env.local`: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_API_URL`, and a commented-out `NEXT_PUBLIC_APP_URL` for production custom domains.

Chapter 9: Database Schema & Migrations — supabase/

The data model is defined by a base schema plus four sequential migrations. In production, only

`002_agentedge_idempotent.sql`, `003_platform_memory.sql`, and `004_market_splits.sql` are actually applied by the migration runner (`scripts/run_migration.py`) — `001_agentedge.sql` was superseded by the idempotent `002` rewrite, and the base `schema.sql` is applied manually once, up front, via the Supabase SQL Editor.

supabase/schema.sql

Purpose: The base Supabase schema for the "EdgeBet" platform — the original core schema that predates the AgentEdge agent-infrastructure migrations, meant to be pasted directly into the Supabase SQL Editor.

Key contents:

- `profiles` — `id` (PK, FK to `auth.users`), `email`, `full_name`, `is_admin`, `is_active`, `created_at`.
 - Function `handle_new_user()` (SECURITY DEFINER) + trigger `on_auth_user_created` on `auth.users` (AFTER INSERT) that auto-inserts a `profiles` row on signup.
 - `invite_codes` — `id`, `code` (UNIQUE), `created_by` (FK profiles), `max_uses`, `current_uses`, `is_active`, `created_at`. Seeded with code `EDGE BET2026` (`max_uses` 10, active).
 - `preferences` — `id`, `user_id` (FK profiles, UNIQUE), `sports` (TEXT[] default `{MLB,NBA,NHL}`), `bet_types` (TEXT[] default `{player_props,straight}`), `risk_level` (default `MEDIUM`), `max_plays` (default 5), `unit_size` (default 50), `include_parlays` (default false), `notification_email`, `updated_at`.
 - `cards` — `id`, `user_id` (FK profiles), `date`, `slate_grade`, `slate_note`, `plays` / `leans` / `quick_reads` / `pass_notes` (JSONB arrays), `raw_card` (JSONB), `created_at`; UNIQUE(`user_id`, `date`).
 - `bets` — `id`, `user_id`, `card_id` (FK cards), `date`, `sport`, `game`, `bet`, `market`, `odds` (INTEGER), `book`, `units` (default 2), `confidence`, `result` (default `pending`), `units_result` (default 0), `post_slate_tag`, `notes`, `created_at`.
 - `agent_memory` — `id`, `user_id` (FK profiles, UNIQUE), `stats` (JSONB), `updated_at`.
 - RLS enabled on `profiles`, `preferences`, `cards`, `bets`, `invite_codes`, `agent_memory`; policies restrict SELECT/UPDATE/ALL to `auth.uid() = <owner column>`, except `invite_codes` which allows anyone to SELECT.
- How it works:** This is the foundational multi-tenant schema: each authenticated user gets a `profiles` row automatically via trigger, personal `preferences`, daily `cards` (structured betting recommendations), and individual `bets` records with pending/settled results tracked via `result` / `units_result`. Invite codes gate signup.

supabase/migrations/001_agentedge.sql

Purpose: First AgentEdge migration, adding the per-user autonomous agent infrastructure on top of the base `schema.sql`. Superseded in practice by `002_agentedge_idempotent.sql`.

Key contents:

- `agent_instances` — one row per user: `status` (default `pending_setup`), `mode` (default `scanning`), `bankroll_starting` / `bankroll_current` (default 1000), `unit_pct` (default 0.02), `max_daily_pct` (default 0.06), `units_at_risk` (default 0), `subscription_tier` (default `beta`), `last_active_at`, `last_scan_at`, `setup_completed_at`, `created_at`.
- `agent_episodes` — decision/live-feed log: `id`, `user_id`, `timestamp`, `trigger_type` (default `scheduled_scan`), `trigger_payload` (JSONB), `episode_type` (default `observation`), `title`, `reasoning`,

`action_payload` (JSONB), `outcome`, `lesson`, `created_at`.

- `agent_beliefs` — learned insights: `id`, `user_id`, `category` (default `general`), `belief`, `confidence` (default 0.5), `evidence_count` (default 1), `last_validated`, `created_at`.
 - `agent_hypotheses` — watch-list theses not yet acted on: `id`, `user_id`, `sport`, `game`, `market`, `player`, `thesis`, `status` (default `watching`), `expires_at`, `created_at`, `updated_at`.
 - `market_snapshots` — shared perception layer: `id`, `sport_key`, `snapshot` (JSONB), `captured_at`.
 - Alters `preferences`: adds `bankroll_starting`, `unit_pct`, `max_daily_pct`; changes default `sports` array to `{MLB,NBA,NHL,NFL,WC}`.
 - Enables RLS on the four new agent_ tables with *SELECT-only "own row" policies (writes are backend/service-role only)*.
- How it works:** Models each user's betting agent as a stateful entity with a bankroll/risk parameters, a chronological reasoning trail, accumulated learned beliefs with confidence scores, and speculative watches. `market_snapshots` is shared across users (keyed by sport, not user).

[supabase/migrations/002_agentedge_idempotent.sql](#)

Purpose: A safe-to-rerun version of migration 001, used because 001's non-idempotent `CREATE POLICY` statements fail on rerun. This is the version actually referenced by `scripts/run_migration.py`'s `MIGRATION_ORDER` and applied in production.

Key contents: Identical table/index/column definitions to 001, plus it also (re)creates `agent_memory` (same shape as `schema.sql`) here. RLS: enables RLS on all five tables and uses `DROP POLICY IF EXISTS ... ; CREATE POLICY ...` pairs for every policy so reruns never fail on duplicate policy names.

How it works: Functionally a merge of `schema.sql`'s `agent_memory` and `001_agentedge.sql`'s new tables, rewritten entirely with `IF NOT EXISTS` / `DROP POLICY IF EXISTS` guards for idempotency.

[supabase/migrations/003_platform_memory.sql](#)

Purpose: Adds a single global (cross-user) "platform brain" table that aggregates collective performance memory across all users' agents, used for prompt injection into agent reasoning.

Key contents: `platform_memory` — `key` (TEXT PRIMARY KEY, default `'global'`), `stats` (JSONB, default `{}`), `updated_at`. Seeds one row via `INSERT ... ON CONFLICT (key) DO NOTHING`.

How it works: A singleton row (key-value store restricted to one key today). No RLS — "Service role writes; no user-facing RLS needed for prompt injection via backend," meaning only the backend reads/writes it.

[supabase/migrations/004_market_splits.sql](#)

Purpose: "Phase 2" migration adding storage for external betting-market splits data (public bet %, money %, sharp money indicators) sourced from third-party odds/sharp-money providers.

Key contents: `market_splits` — `id`, `sport_key`, `event_id`, `home_team`, `away_team`, `market`, `public_bet_pct` (DECIMAL), `public_money_pct` (DECIMAL), `sharp_indicator` (TEXT), `source` (TEXT NOT NULL), `raw` (JSONB), `captured_at`. RLS is enabled but no policies are defined, so only the service role can read/write.

How it works: Populated by an "Action Network / SportsDataIO / OddsJam sync job." A shared (not per-user) table analogous to `market_snapshots`, giving the agent additional sharp-vs-public sentiment signal per event/market.

Chapter 10: CI/CD, Automation & Operational Docs

[.github/workflows/production-setup.yml](#)

Purpose: GitHub Actions workflow that runs Supabase SQL migrations and seeds the backend's "learning loop" for production — the CD step that provisions/updates the database schema and initial data whenever migrations or setup scripts change.

Key contents:

- Triggers: `workflow_dispatch` (manual) and `push` to `main` filtered to paths `supabase/migrations/**`, `backend/scripts/production_setup.py`, `scripts/run_migration.py`, and the workflow file itself.
- Job `setup` on `ubuntu-latest`: checkout → set up Python 3.12 (pip cache) → install `backend/requirements.txt` + `psycpg2-binary` → "Run Supabase migrations" (skips with a warning if `SUPABASE_DB_URL` unset; otherwise runs `python scripts/run_migration.py`, tolerating failure) → "Seed learning loop" (`python scripts/production_setup.py` from `backend/`, with `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`, `TIMEZONE=America/Denver`).

How it works / notes: Required secrets: `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`; optional `SUPABASE_DB_URL` (needed only for the raw SQL migration step). The migration step is deliberately non-fatal so idempotent replays don't break the pipeline.

[.github/workflows/sync-sheets.yml](#)

Purpose: Scheduled workflow that exports all bets (including pending ones) from Supabase into a Google Sheet for tracking/visibility outside the app.

Key contents: Triggers: `workflow_dispatch` and two `schedule` crons (`"5 16 * * *"`, `"5 17 * * *"` UTC) covering 10:05 AM Mountain Time across MDT/MST — run after the WC card (8:50) and MLB (9:35) jobs on Railway. Job installs deps and runs `python sync_sheets.py` from `backend/` with `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`, `GOOGLE_SHEET_ID`, `GOOGLE_SHEETS_CREDENTIALS_JSON`, `GOOGLE_SHEETS_SYNC_EMAIL`.

How it works / notes: Acts as a redundant/backup sync to Railway's own APScheduler-driven sync, guarding against that job failing or not running.

[.github/workflows/wc-card.yml](#)

Purpose: GitHub Actions workflow that generates and emails the daily World Cup betting card — the primary scheduled trigger for that feature (Railway's own scheduler is the backup).

Key contents:

- Triggers: `workflow_dispatch` (inputs: `date`, `email`, `no_email`, `force`) and `schedule` with two crons (`"50 14 * * *"`, `"50 15 * * *"` UTC) to hit 8:50 AM Mountain Time across DST.
- Job `generate-card`: checkout → Python 3.12 setup → "Resolve card date" → "Verify 8:50 AM Mountain Time" (dedupes the two UTC cron slots depending on DST, sets `skip=true/false`) → "Run World Cup card" (`python run_world_cup_card.py --date $DATE [...] from backend/`, with a large env block: `ANTHROPIC_API_KEY`, `ODDS_API_KEY`, `SGO_API_KEY`, `EMAIL_FROM`, `EMAIL_SMTP_*`, `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`, `GOOGLE_SHEET_ID`, `GOOGLE_SHEETS_CREDENTIALS_JSON`, `GOOGLE_SHEETS_SYNC_EMAIL`, `WC_CARD_USER_EMAIL`, `TIMEZONE=America/Denver`).

How it works / notes: The dual-cron + MT-window-check pattern works around GitHub Actions cron only supporting UTC,

ensuring the card fires once a day at true 8:50 AM Mountain Time year-round. Card generation is idempotent per day (skips duplicates unless `--force`).

`docs/CUSTOM_DOMAIN.md`

Purpose: Step-by-step runbook for migrating the AgentEdge frontend from the default Vercel URL to a branded custom domain (e.g., `app.edgebet.com`).

Key contents: Five sections: (1) Vercel domain attachment (CNAME record, HTTPS propagation); (2) Vercel env vars for Production (`NEXT_PUBLIC_APP_URL`, `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_API_URL`); (3) Supabase Auth URL Configuration (Site URL + Redirect URLs); (4) Railway backend CORS (`FRONTEND_URL`); (5) Verification checklist. Ends with a troubleshooting table.

How it works / notes: Confirms concrete infra identifiers: Supabase project `nlfalrpuspdezfnlakrv`, Railway backend `sportsagent-production.up.railway.app`, current Vercel domain `sports-agent-phi.vercel.app`.

`docs/sessions/README.md`

Purpose: Explains the convention for optional per-session detail files (`YYYY-MM-DD.md`) used to log development/agent work sessions, tied to the root `CLAUDE.md` "Session Log."

`scripts/run_migration.py`

Purpose: Standalone CLI that applies AgentEdge Supabase SQL migrations in order against a direct Postgres connection; invoked by `production-setup.yml`.

Key contents: `MIGRATION_ORDER` = (`"002_agentedge_idempotent.sql"`, `"003_platform_memory.sql"`, `"004_market_splits.sql"`) — note `001_agentedge.sql` and `schema.sql` are intentionally excluded/superseded. `parse_postgres_url` manually parses a `postgresql://` URI (handling URL-encoded passwords); `connect_postgres` connects via `psycopg2` with `sslmode="require"`; `main()` reads `SUPABASE_DB_URL` / `DATABASE_URL`, executes each migration file's full SQL text with autocommit, skipping missing files.

How it works / notes: Designed to be safely re-run given the migrations' own idempotent guards.

`scripts/set_railway_env.sh`

Purpose: Bash helper to bulk-push AgentEdge backend environment variables from a local `backend/.env` file to the linked Railway service via the Railway CLI.

Key contents: Requires prior `railway login` / `railway link`; sources `.env`; runs one `railway variables set` populating odds-provider quota/interval knobs (`TOA_MONTHLY_QUOTA`, `TOA_RESERVE`, `SGO_MONTHLY_OBJECTS`, `ODDS_POLL_INTERVAL_MINUTES`, `AGENT_SCAN_INTERVAL_MINUTES`, `SNAPSHOT_MAX_AGE_MINUTES`, etc.) plus pass-through secrets (API keys, Supabase, email, Sheets).

How it works / notes: Falls back to sensible defaults via bash `${VAR:-default}` syntax for tuning knobs while requiring real secrets from `.env`.

`railway.toml` (repo root)

Purpose: Alternative Railway deployment config used when the Railway service's Root Directory is left at `/` rather than `backend/`. Points the Dockerfile build at `backend/Dockerfile`, limits rebuild-triggering file watches to `backend/**`, healthchecks `/health` (30s timeout), restarts `ON_FAILURE`.

`.gitignore`

Purpose: Standard ignore rules for the mixed Python + Next.js monorepo: `*.bak`, `.env`, `__pycache__`, `*.pyc`, `*.pyo`, `.vercel`, `node_modules/`, `.next/`.

Chapter 11: Test Coverage

`backend/tests/test_claude_config.py`

Purpose: Covers Claude API cost-estimation and cost-control config flags in `esm.claude_config`.

What it tests: `estimate_call_cost_usd` computes correct blended cost for 10k input / 2k output tokens (\$0.05–\$0.07 range); cost estimate stays positive when `cache_read_input_tokens` is present; `AGENT_SCANS_ENABLED` env var parses correctly (`"false"` → `False`) after module reload.

`backend/tests/test_dual_brain_memory.py`

Purpose: Tests the platform-level ("secondary brain") aggregation helpers in `learning.memory`, ensuring cross-user stats are anonymized.

What it tests: `_weak_markets_platform` flags a poor-record market and requires a `min_bets` threshold, never including individual bet text; `_compute_platform_stats` correctly counts distinct `active_users` / `total_bets`, sets `scope: "platform"`, and omits per-user `recent_losses`.

`backend/tests/test_grader_soccer.py`

Purpose: Unit tests for soccer bet-grading parsing/resolution logic in `services.grader`, plus a spot-check of unit payout math.

What it tests: `_parse_game_teams` handles "vs"/"@"/"v" separators with parenthetical group labels; `_resolve_soccer_outcome` correctly grades DNB (win/loss/push on draw), 3-way ML, Total Goals, and BTTS markets; parser helpers for DNB/ML/total/BTTS teams; `_team_matches` fuzzy matching; `calculate_win_units` sanity checks.

`backend/tests/test_learning_hooks.py`

Purpose: Tests agent learning-loop helper functions in `agent.bankroll` and `agent.memory_store` (episode lookup, lesson grading, prompt formatting).

What it tests: `apply_bet_result` / `compute_unit_size` increment/decrement bankroll correctly; `_grade_lesson` produces "Win"/"model miss"/"Push" text appropriately; `_find_position_episode` locates an open position episode by `bet_id` or bet-text fallback, skipping already-graded episodes; `format_hypotheses_for_prompt` / `format_recent_episodes_for_prompt` render expected content and exclude non-position episodes.

`backend/tests/test_market_intelligence.py`

Purpose: Tests line-movement/market-intelligence analytics in `esm.market_intelligence` (steam detection, reverse line movement, public betting splits).

What it tests: `analyze_game_lines` detects moneyline steam moves and reverse line movement with sharp-money flags; `format_intelligence_for_prompt` renders expected flag names; `sharp_action_label` produces "Steam"/"Reverse line" labels; public splits set `data_quality: "snapshot_plus_splits"` and contrarian/public-heavy flags correctly.

backend/tests/test_sheets_sync.py

Purpose: Tests Google Sheets export/row-building helpers in `services.sheets_sync` without real API calls.

What it tests: `_calc_record` computes W-L-P record and net units correctly;

`_build_overall_record_rows` / `_build_by_sport_rows` produce correct header/aggregate rows; `BETS_HEADERS` has exactly 19 columns; `BY_SPORT_HEADERS` starts with "Sport".

backend/tests/test_soccer_dnb_odds.py

Purpose: Tests Draw No Bet (DNB) odds validation/juice-ceiling enforcement for World Cup official plays in `esm.soccer_odds`.

What it tests: `within_juice_ceiling` accepts up to -150, rejects juicier lines; `parse_dnb_team` / `posted_dnb_odds` extract correct team/price; `validate_wc_official_plays` removes a DNB play exceeding -150 (appending a pass note) and corrects posted odds when within ceiling.

backend/tests/test_tier1_learning.py

Purpose: Tests Tier 1 learning pipeline-split memory formatting (`learning.memory`) and bankroll-replay backfill (`agent.bankroll_backfill`).

What it tests: `_pipeline_key` normalizes `post_slate_tag` values; `_aggregate_bets` computes wins/losses/net units and per-market breakdowns; `_format_user_memory` / `_format_platform_memory` prioritize active pipeline and anonymize platform data; `get_performance_context` orders user memory before platform memory; `replay_bankroll_for_user` replays historical bets to a correctly compounded bankroll.

backend/tests/test_unit_tracker.py

Purpose: Tests 3%-of-bankroll unit sizing, daily risk caps, and World Cup bet exclusion logic in `agent.bankroll` / `agent.unit_tracker`.

What it tests: `compute_unit_size` returns 3% of bankroll; `compute_max_daily_units` caps daily units correctly (e.g. 2 units on a 6%/3% \$1000 bankroll); `compute_bankroll_summary` integrates unit size/max/remaining; `is_wc_bet` identifies World Cup bets via tag or legacy notes marker; `major_league_sport_keys` excludes `soccer_fifa_world_cup`.

backend/tests/test_units.py

Purpose: Core unit-conversion and P/L math tests for `services.units`.

What it tests: `calculate_win_units` correctly converts stake+odds to profit for negative/positive/fractional American odds; `calculate_units_result` returns full negative stake on a loss; pushes always return 0.0; `normalize_units_result` corrects a mismatched stored value; `aggregate_record` computes W-L-P record, net units, and units risked/won/lost across a sample.

Chapter 12: Environment Variables & Deployment Reference

Backend environment variables

ANTHROPIC_API_KEY	Claude API key
SUPABASE_URL	Supabase project URL
SUPABASE_SERVICE_KEY	Supabase service-role key (bypasses RLS; used by backend/db.py)
SUPABASE_ANON_KEY	Supabase anon key (used for user-token auth checks)
SUPABASE_DB_URL	Direct Postgres connection string (migrations only)
ODDS_API_KEY	The Odds API (TOA) key
SGO_API_KEY	SportsGameOdds (SGO) key
ODDS_PRIMARY_SOURCE	"sgo" (default) or "toa"
FRONTEND_URL	Comma-separated list of allowed CORS origins
TIMEZONE	Default "America/Denver"
CLAUDE_MODEL	Default "claude-sonnet-4-6"
AGENT_SCANS_ENABLED	Feature flag / cost kill-switch for continuous agent scans
ODDS_POLL_INTERVAL_MINUTES	Shared market-poller cadence (default 360)
AGENT_SCAN_INTERVAL_MINUTES	Per-user agent scan cadence (default 180)
SNAPSHOT_MAX_AGE_MINUTES	Cache staleness threshold (default 360)
UNIT_PCT / MAX_DAILY_PCT	Default unit-sizing percentages (3% / 6%)
WC_CARD_ENABLED / WC_CARD_USER_EMAIL	World Cup pipeline config
EMAIL_FROM / EMAIL_SMTP_HOST/PORT/USER/PASS / SENDGRID_API_KEY	Email delivery
GOOGLE_SHEET_ID / GOOGLE_SHEETS_CREDENTIALS_JSON / GOOGLE_SHEETS_SYNC_EMAIL	Sheets export

Frontend environment variables

NEXT_PUBLIC_SUPABASE_URL	
NEXT_PUBLIC_SUPABASE_ANON_KEY	
NEXT_PUBLIC_API_URL	Backend base URL (default http://localhost:8000)
NEXT_PUBLIC_APP_URL	Public frontend URL (Vercel custom domain support)

Production URLs

Service	URL
Frontend (Vercel)	https://sports-agent-phi.vercel.app
Backend (Railway)	https://sportsagent-production.up.railway.app
Supabase project	nlfalrpuspdezfnlakrv.supabase.co

Running locally

```
# Backend
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000

# Frontend
cd frontend
npm install
npm run dev
```

Document generated by Claude Code from a full read-through of every tracked file in `austin2134-cell/sportsagent` as of 2026-07-06.